python

Graphical User Interface (GUI)

Introduction

- We now that where user uses commands or characters to interact with application are generally known as CUI (Character User Interface) or Console applications. Ex. MS-DOS
- Another way to interact with application through graphics, images or pictures are known as GUI (Graphical User Interface). In this case used doesn't remember any commands.
- Advantages of GUI are as follows
  - It is user friendly
  - It adds beauty to any application by adding pictures, colors, menus, animations etc.
  - It simulate real word object using GUI
  - GUI helps to create graphical components like push button, radio button, textboxes etc.

GUI in Python

- Python provides various options for developing graphical user interfaces (GUIs).
- Most important are listed below.
  - Tkinter – Tkinter is the Python interface to the Tk GUI toolkit shipped with Python.
  - wxPython – This is an open-source Python interface for wxWindows http://wxpython.org.
  - JPython – JPython is a Python port for Java which gives Python scripts seamless access to Java class libraries on the local machine http://www.jython.org.
  - Qt-There are bindings available for the Qt toolkit (using either PyQt or PySide) and for KDE (PyKDE4). PyQt is currently more mature than PySide, but you must buy a PyQt license from Riverbank Computing if you want to write proprietary applications. PySide is free for all applications.
- There are many other interfaces available, which you can find them on the net.

Tkinter Module

- Python offers **tkinter** module to create GUI program.
- Tkinter is the standard GUI library for Python.
- Tkinter represents toolkit interface for GUI
- Python when combined with Tkinter provides a fast and easy way to create GUI applications.
- Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit.
- Tkinter (pronounced tea-kay-inter) is the Python interface to Tk, the GUI toolkit for TCL/Tk.
- TCL (Tool Command Language and pronounced as tickle) is a popular scripting language in the domains of embedded applications, testing, prototyping, and GUI development.
- On the other hand, Tk is an open source, multi-platform widget toolkit that is used by many different languages to build GUI programs.
- Tkinter is suitable for application to a wide variety of areas, ranging from small desktop applications to use in scientific modeling and research endeavors across various disciplines.

Tkinter Module

- The Tk widget library originates from the Tool Command Language (Tcl) programming language.
- Tcl and Tk were created by John Ousterman while he was a professor at Berkeley in the late 1980s as an easier way to program engineering tools being used at the university.
- Because of its speed and relative simplicity, Tcl/Tk rapidly grew in popularity among academic, engineering, and Unix programmers.
- The features that make Tkinter a great choice for GUI programming include the following:
  - It is simple to learn (simpler than any other GUI package for Python)
  - Relatively little code can produce powerful GUI applications
  - Layered design ensures that it is easy to grasp
  - It is portable across all operating systems
  - It is easily accessible, as it comes pre-installed with the standard Python distribution

Tkinter Module

- Tkinter is a Python interface to the Tk GUI library and has been a part of the Python standard library since 1994 with the release of Python version 1.1, making it the de facto GUI library for Python.
- Documentation for Tkinter, along with links for further study, can be found in the standard library documentation at https://docs.python.org/3/library/tkinter.html.
- it also has the following
  - It's in the standard library:
  - It's stable
  - It's only a GUI toolkit
  - It's simple and no-nonsense
- Disadvantages
  - Look and feel
  - Complex widgets

Tkinter Module:-Installation

- Tkinter is included in the Python standard library for the Windows and macOS distributions.
- That means that, if you have Python on these platforms, you don't need to do anything to install Tkinter.
- Most of the time, you will not need to install Tkinter as long as you have Python installed.
- To check, open an instance of the interactive interpreter and type **import tkinter.**
- If you don't see an error, then Tkinter is already installed and you are ready to go!
- To check your Tkinter version, open an interactive Python prompt and type the following:
  - ***import tkinter***
  - ***tkinter.TkVersion***
- To check whether you have the correct Tkinter version on your Python installation, type the following commands in your IDLE or interactive shell:
  - ***tkinter._test()***

Tkinter Module:-Installation

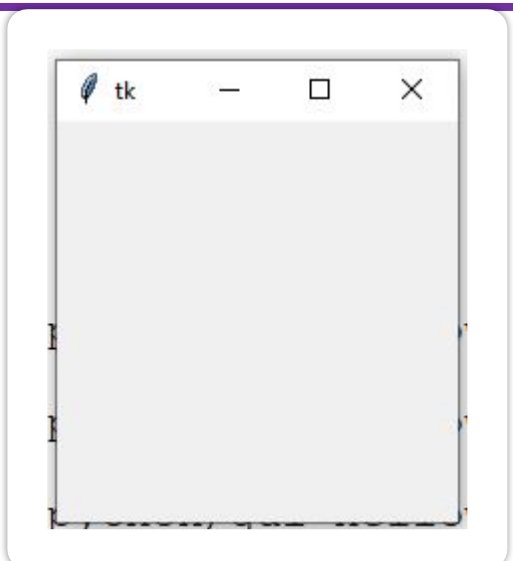
- This should make a window pop up.
- The first line in the window reads ***"This is Tcl/Tk version 8.6".***
- Make sure that it is not 8.5 or any earlier version, as Version 8.6 is a vast improvement over its previous versions.
- You are ready to code Tkinter GUI applications if your version test confirms it as Tcl/Tk version 8.6.
- There are several ways to import Tkinter
  - ***import tkinter***
  - ***import tkinter as tk***
- The tk alias is convenient and easy to type. It is commonly used by many developers to import Tkinter.

GUI in Python

- Following are general steps involving in basic GUI programming
  1. First we should create the root window. The root window is top level window which provides space on screen where we can display different component.
  2. In root window we then place frame or canvas which is useful to allocate space for our use. Here frame/canvas are child window of root window
  3. Generally we use canvas for displaying drawings like lines, circled, rectangles, shapes etc. and frames are used to displays components (widgets) like push button, text box, radio buttons etc.
  4. When the user clicks on widgets lie push buttons, we need to handle that events i.e we have to responds to events by performing the desired tasks.
-

- **Root Window**
  - To display graphical output we need space on the screen.
  - This space initially allocated to every GUI program is called as top level/root window.
  - The root window is highest level GUI component in any tkinter application where we can display different component.
  - The root window will be created by creating object of TK class.
  - The root window has title bar showing min, max and resize button.

```
from tkinter import *
root = Tk()
root.mainloop()
```



- **Root Window**
  - The first line imported all (*) the classes, attributes, and methods of Tkinter into the current workspace.
  - The second line created an instance of the tkinter.Tk class which creates "root" window.
  - The third line executed the mainloop (that is, the event loop) method of the root object. The mainloop method is what keeps the root window visible. If you remove the third line, the window created in line 2 will disappear immediately as soon as the script stops running. This will happen so fast that you will not even see the window appearing on your screen. Keeping the mainloop method running also lets you keep the program running until you press the close button, which exits the main loop.
  - Tkinter also exposed the mainloop method as tkinter.mainloop(). So, you can even call mainloop() directly instead of calling root.mainloop().

- **Root Window**
  - Following are some important methods used for root window

Method	Description
title()	method to change the title of the window.
geometry()	method to change the size and location of the window.
resizable()	method to specify whether a window can be resizable horizontally or vertically.
attributes('-alpha',0.5)	to set the transparency for the window
attributes('-topmost', 1)	to make the window always on top.
lift() and lower()	methods to move the window up and down of the window stacking order.
iconbitmap()	method to change the default icon of the window.

- **Root Window**

```
import tkinter as tk


root = tk.Tk()
root.title('Tkinter Window Demo')
root.geometry('600x400+50+50')
root.resizable(False, False)
root.attributes('-alpha', 0.8)
root.attributes('-topmost', 1)
root.iconbitmap('f:/python/icon_1.ico')
root.mainloop()
```

- **Containers**
  - Container is space that displays the output to the user.
  - There are two major containers used
    - Canvas-Generally useful for different shapes
    - Frame-Generally useful to display widgets
  - After creating root window, we have to create space i.e. container.

- **Canvas**
  - It is rectangular area which are generally used for drawing pictures like shapes, lines etc.
  - To create canvas we need to create object of canvas class as
  - ***c=Canvas(root, bg="blue", height=500, width=600, cursor="pencil")***
  - Here c is the canvas class object
  - root is name of parent window
  - Height and width represent height and width of canvas in pixels
  - bg represent background color such as their names(ex.-red, blue, black etc)
  - cursor represent shape of cursor in the canvas. Possible values are arrow, "arrow“, "circle“, "clock“, "cross“, "dotbox“, "exchange“, "fleur“, "heart“, "heart“, "man“, "mouse“, "pirate“, "pencil“, "plus“, "shuttle“, "sizing“, "spider“, "spraycan“, "star“, "target“, "tcross“, "trek“, "watch“ etc.

- **Canvas**
  - Once canvas is create, it should be added to the root window then and then it will be visible.
  - This can be done using pack() method as ***c.pack()***
  - After canvas is create, we can draw any shape on the canvas.

Method	Description
create_line(x1, y1, x2, y2, ..., options = ...)	Used to create Line
create_oval(x1, y1, x2, y2, options = ...)	It is used to create a oval, pieslice and chord.
create_rectangle(x1, y1, x2, y2, options = ...)	It is used to create rectangle and square.
create_arc(x1, y1, x2, y2, options = ...)	This is used to create an arc.
create_polygon(coordinates, options = ...)	This is used to create any valid shapes.
create_text(x, y, option, ...)	You can display one or more lines of text on a canvas
create_image(x, y, option, ...)	To display a graphics image on a canvas

- **Canvas**

```
from tkinter import *
root = Tk()
c=Canvas(root,bg="blue", height=600, width=900, cursor="pencil")
obj=c.create_line(50,50,200,50,200,150, width=4, fill="white")
obj=c.create_oval(100,100,400,300, width=5, fill="yellow", outline="red", activefill="green")
obj=c.create_polygon(10,10,200,200,300,200, width=3, fill="green", outline="red", smooth=1, activefill="red")
obj=c.create_rectangle(300,200,700,600, width=2, fill="gray", outline="black",  activefill="yellow")
obj=c.create_text(500,100, text="This is Sample Canvas", fill="yellow", activefill="green")
fnt=("Times",40,"bold italic underline")
obj=c.create_text(300,400, text="This is Sample Canvas",font=fnt, fill="yellow", activefill="green")
c.pack()
root.mainloop()
```

- **Canvas**
  - *obj=c.create_line(50,50,200,50,200,150, width=4, fill="white")*
    - Creates line with corresponding points (50,50), (200,50) and (200,150)
    - Width specifies width of line (default is 1)
    - fill specifies color of line
    - create_line() method return an id number
  - *obj=c.create_oval(100,100,400,300, width=5, fill="yellow", outline="red", activefill="green")*
    - Creates oval in rectangular area defined by top left coordinates (100,100) and bottom lower coordinates (400,300)
    - if rectangle has same height and width then it become circle
    - Outline represents the color to be used for border
    - activefill represents the color to be filled when mouse is placed on the oval

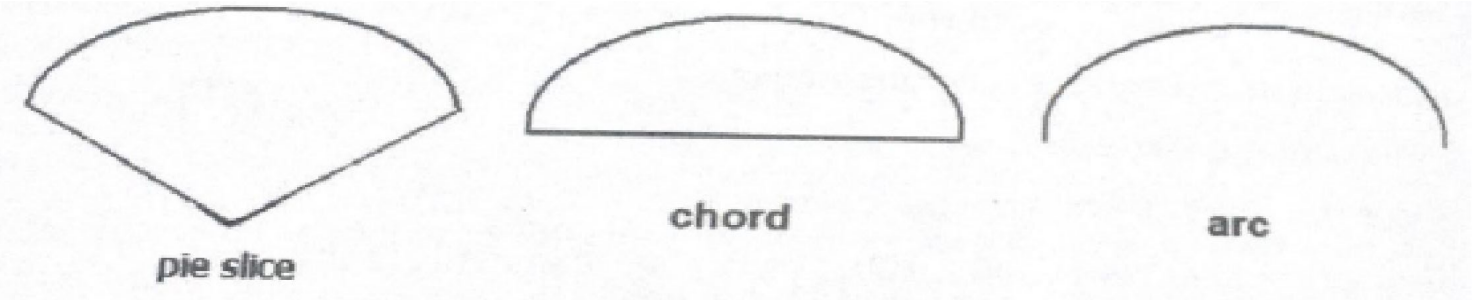
- **Canvas**
  - *obj=c.create_polygon(10,10,200,200,300,200, width=3, fill="green", outline="red", smooth=1, activefill="red")*
    - Creates polygon with points (10,10), (200,200) and (300,200) and then last point connected to first point i.e. (10,10)
    - In option smooth 0 indicates polygon with sharp edges and 1 indicates smooth edges.
  - *obj=c.create_rectangle(300,200,700,600, width=2, fill="gray", outline="black", activefill="yellow")*
    - Creates rectangle by top left coordinates (300,200) and bottom lower coordinates (700,600)

- **Canvas**
  - *obj=c.create_text(500,100, text="This is Sample Canvas", fill="yellow", activefill="green")*
    - Display some text mentioned in text attribute at the point (500,100)
  - *fnt=("Times",40,"bold italic underline")*
    - This is used to create font object.
  - *obj=c.create_text(300,400, text="This is Sample Canvas",font=fnt, fill="yellow", activefill="green")*
    - Font attribute is used to sent font for given text

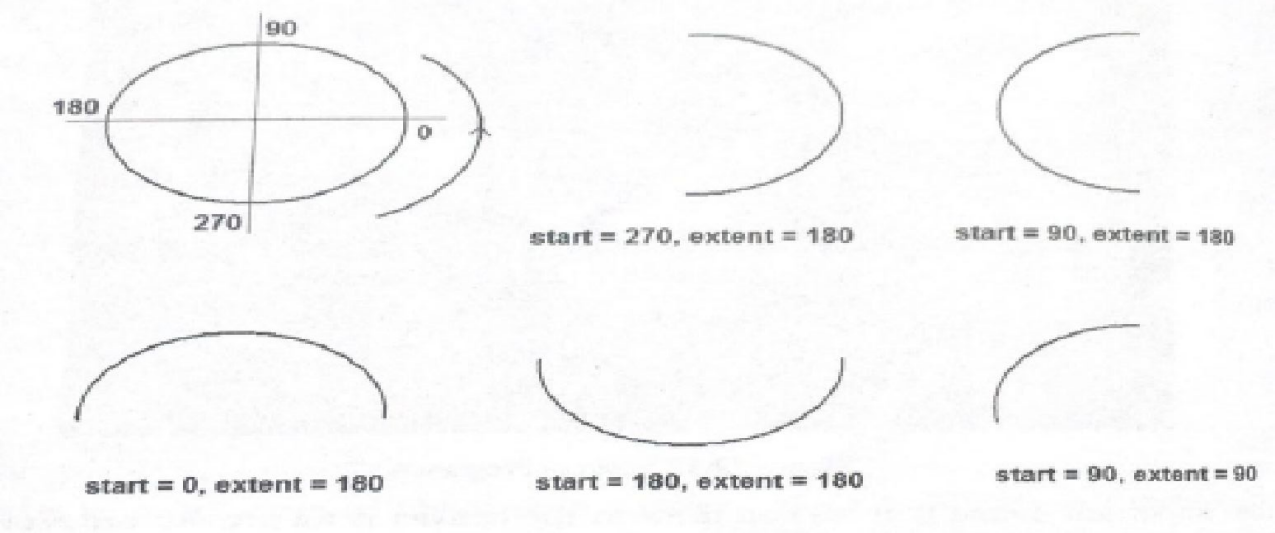
- **Canvas**

```
from tkinter import *


root = Tk()
c=Canvas(root,bg="gray", height=700, width=1200)
obj=c.create_arc(100,100,400,300, width=3, start=270, extent=180, outline="red", style="arc")
obj=c.create_arc(500,100,800,300, width=3, start=90, extent=180, outline="red", style="arc")
obj=c.create_arc(100,400,400,600, width=3, start=0, extent=180, outline="blue", style="arc")
obj=c.create_arc(500,400,800,600, width=3, start=180, extent=180, outline="blue", style="chord")
obj=c.create_arc(900,400,1200,600, width=3, start=90, extent=90, outline="black", style="arc")


c.pack()
root.mainloop()
```

- **Canvas**
  - *obj=c.create_arc(100,100,400,300, width=3, start=270, extent=180, outline="red", style="arc")*
    - Can drown arc in the canvas
    - Arc is part of ellipse or circle
    - Width represent width of arc in pixel
    - Start represent starting degree of arc
    - Extent represent extend arc drawing up to the degree. (ex. Here start is 270 and extent is 180 i.e. extent up to 270+180=450 i.e. 450-3600=90)
    - Style option can be arc, pieslice or chord.



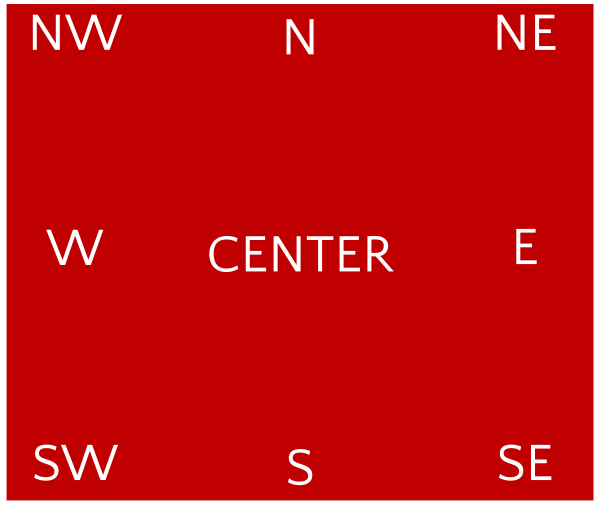
- **Canvas**
  - *obj=c.create_arc(100,100,400,300, width=3, start=270, extent=180, outline="red", style="arc")*
    - As mentioned start represent an angle of the arc where it has to start
    - Extent represent the angle further which the arc should extend
    - These angle should be taken in clock wise direction and 3 O' clock position as 0 degree, 12 O' clock position as 90 degree, 9 O' clock position as 180 degree and 6 O' clock position as 270 degree.



- **Canvas**

```
from tkinter import *


root = Tk()
c=Canvas(root,bg="gray", height=700, width=1200)


file1=PhotoImage(file="f://python/cat.gif")
file2=PhotoImage(file="f://python/cat1.gif")
obj=c.create_image(800,100,anchor=NE, image=file1, activeimage=file2)
c.pack()
root.mainloop()
```

- **Canvas**
  - *file1=PhotoImage(file="f://python/cat.gif")*
  - *obj=c.create_image(800,100,anchor=NE, image=file1, activeimage=file2)*
    - We can display image in canvas using create_image method.
    - We should load image into the file using PhotoImage class.
    - The image can be placed in any direction from the points indicated by anchor
    - Possible values for anchor attributes are as:- NW, N, NE, E, SE, S, SW, W and CENTER



- **Frame**
  - A frame is similar to canvas the represent a rectangular area where mostly widgets can be displayed.
  - Actually root window is one frame.
  - To create frame we can create object of Frame class as
    - ***f=Frame(root, height=500, width=600, bg="gray", cursor="cross")***
    - Here f is object of Frame class
    - root is parent window
    - Height and width are height and width of frame in pixel
    - bg represent background color
    - cursor represent type o cursor to be displayed in the frame
  - Once frame is create it should be added in root window using pack() method as
    - f.pack()

- **Frames**

```
from tkinter import *


root = Tk()
root.title("My Sample Frame")
f=Frame(root, height=500, width=600, bg="#667711", cursor="cross")
f.pack()
root.mainloop()
```

- **Widgets-** The building blocks of GUI programs
  - It is a UI component that is displayed on the screen and can perform desired task.
  - Ex. Push button, check box, radio button etc.
  - We can create widget by using object of specific widget i.e. push button is a widget that is object of Button class.
  - Note the following few important features that are common to all widgets:
    - All widgets are actually objects derived from their respective widget classes.
    - Each widget has a set of options that decides its behavior and appearance. Ex. attributes such as text labels, colors, and font size.
    - To set these attributes, you can set the values directly at the time of creating the widget. Alternatively, you can later set or change the options of the widget by using the .config() or .configure() method. In fact, the .config() method is simply an alias of the .configure() method.

GUI in Python-Widgets

- **Widgets–** The building blocks of GUI programs
  - There are two ways to create widgets in Tkinter.
  - The first way involves creating a widget in one line and then adding the pack() method (or other geometry managers), as follows:
    - ***my_label = Label(root, text="I am a label widget")***
    - ***my_label.pack()***
  - Alternatively, you can write both the lines together, as follows:
    - ***Label(root, text="I am a label widget").pack()***
  - You can either save a reference to the widget created (my_label, as in the first example), or create a widget without keeping any reference to it (as demonstrated in the second example).
  - You should ideally keep a reference to the widget in case the widget has to be accessed later on in the program.
  - If the widget state is supposed to remain static after its creation, you need not keep a reference to the widget.

GUI in Python-Widgets

- **Widgets-** The building blocks of GUI programs
  - Tkinter includes 21 core widgets, which are as follows:

Toplevel widget	Label widget	Button widget
Canvas widget	Checkbutton widget	Entry widget
Frame widget	LabelFrame widget	Listbox widget
Menu widget	Menubutton widget	Message widget
OptionMenu widget	PanedWindow widget	Radiobutton widget
Scale widget	Scrollbar widget	Spinbox widget
Text widget	Bitmap Class widget	Image Class widget

- As a reminder, the syntax for adding a widget is as follows:
- ***Widget-name(its_parent, **its_configuration_options)***

- **Widgets–** The building blocks of GUI programs
  - In general working with widgets takes following steps
    1. Create a widget by creating object of respected widget class.
       - Ex. For push button ***b=Button(f, text='My Button')*** here f is parent frame
    2. When user interact with widget, he will generate an event. (Ex. On clicking button buttonclick event will generate.) such event should be handled by writing function. This is called as event handler or callback handler.

       ```
       def buttonClick(self):
           print("Click event called")
       ```

    3. The clicking event should be linked with callback handler function.

       b.bind('<Button-1>',buttonClick)

    4. These events are continuously monitoring by our program whit the help of loop (event loop)

       root.mainloop()

- **Button Widgets-**
  - Push button is one of the common and most used component in any GUI.
  - It perform some action when clicked
  - Button is create by creating object of Button class
  - ***b=Button(f, text="Click Me", width=15, height=3, bg="gray", fg="red", activebackground="green", activeforeground="white“, command=buttonClick)***
  - Here f represent the parent frame where we add the button
  - text represent text to display on the button
  - height and width represent height and width of button in pixel
  - bg and fg represent background and font color. Here we directly write color name
  - activebackground and activeforeground represent active background and font color.
  - command links the push button with event handler function or we can bind button with function using bind function as
  - ***b.bind(‘<Button-1>’,buttonClick)***

- **Button Widgets-**
  - Push button is one of the common and most used component in any GUI.
  - It perform some action when clicked
  - Button is create by creating object of Button class
  - ***b=Button(f, text="Click Me", width=15, height=3, bg="gray", fg="red", activebackground="green", activeforeground="white“, command=buttonClick)***
  - Here f represent the parent frame where we add the button
  - text represent text to display on the button
  - height and width represent height and width of button in pixel
  - bg and fg represent background and font color. Here we directly write color name
  - activebackground and activeforeground represent active background and font color.
  - command links the push button with event handler function or we can bind button with function using bind function as
  - ***b.bind(‘<Button-1>’,buttonClick)***

• Button Widgets-

Option name	Description
activebackground, activeforeground	Indicates the background, font color of the button at the time when the mouse hovers the button.
bd	This option is used to represent the width of the border in pixels.
bg, fg	This option is used to represent the background and foreground color of the button.
command	The command option is used to set the function call which is scheduled at the time when the function is called.
font	This option indicates the font of the button.
height, width	This option indicates the height and width of the button. This height indicates the number of text lines in the case of text lines and it indicates the number of pixels in the case of images.
image	This option indicates the image displayed on the button.
higlightcolor	This option indicates the highlight color when there is a focus on the button
justify	This option is used to indicate the way by which the multiple text lines are represented. For left justification, it is set to LEFT and it is set to RIGHT for the right justification, and CENTER for the center justification.
Padx, pady	This option indicates the additional padding of the button in the horizontal and vertical direction.
underline	This option is used to underline the text of the button.
Wraplength	In the case, if this option's value is set to a positive number, the text lines will be wrapped in order to fit within this length.
state	This option's value set to DISABLED to make the button unresponsive. The ACTIVE mainly represents the active state of the button.

- **Button Widgets-**

```
from tkinter import *
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600)
        self.f.propagate(0)
        self.f.pack()
        self.b=Button(self.f,text="Click Me", width=15, height=3, bg="gray", fg="red", activebackground="green",
activeforeground="white", command=self.buttonClick)
        self.b.pack()
    def buttonClick(self):
        print("You have clicked me")

root=Tk()
mb=MyButton(root)
root.mainloop()
```

- **Button Widgets-**
  - When we create several buttons, we need to know which button is clicked by the user
  - In this situation we have to call event handler function by passing some argument that represent which button is clicked.
  - Here we are using the lambda expression to pass the argument (ex. 1,2..) to the buttonClick() method as
  - ***Command=lambda:buttonClick(arg1,arg2,….)***
  - ***Ex. b=Button(f, text="Click Me", width=15, height=3, command=lambda:buttonClick(1))***
  - We can also set different properties using following ways
  - ***f[“bg”]=“red” or b[“height”]=500***
  - By default, the command button binds to the left-click and the space bar. It does not bind to the return key.

- **Button Widgets-**

```
from tkinter import *
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600)
        self.f.propagate(0)
        self.f.pack()
        self.b1=Button(self.f,text="Red", width=15, height=3,
bg="gray", fg="red", activebackground="green",
activeforeground="white", command=lambda:self.buttonClick(1))

        self.b2=Button(self.f,text="Green", width=15, height=3,
bg="gray", fg="red", activebackground="green",
activeforeground="white", command=lambda:self.buttonClick(2))

       self.b3=Button(self.f,text="Blue", width=15, height=3,
bg="gray", fg="red", activebackground="green",
activeforeground="white", command=lambda:self.buttonClick(3))
```

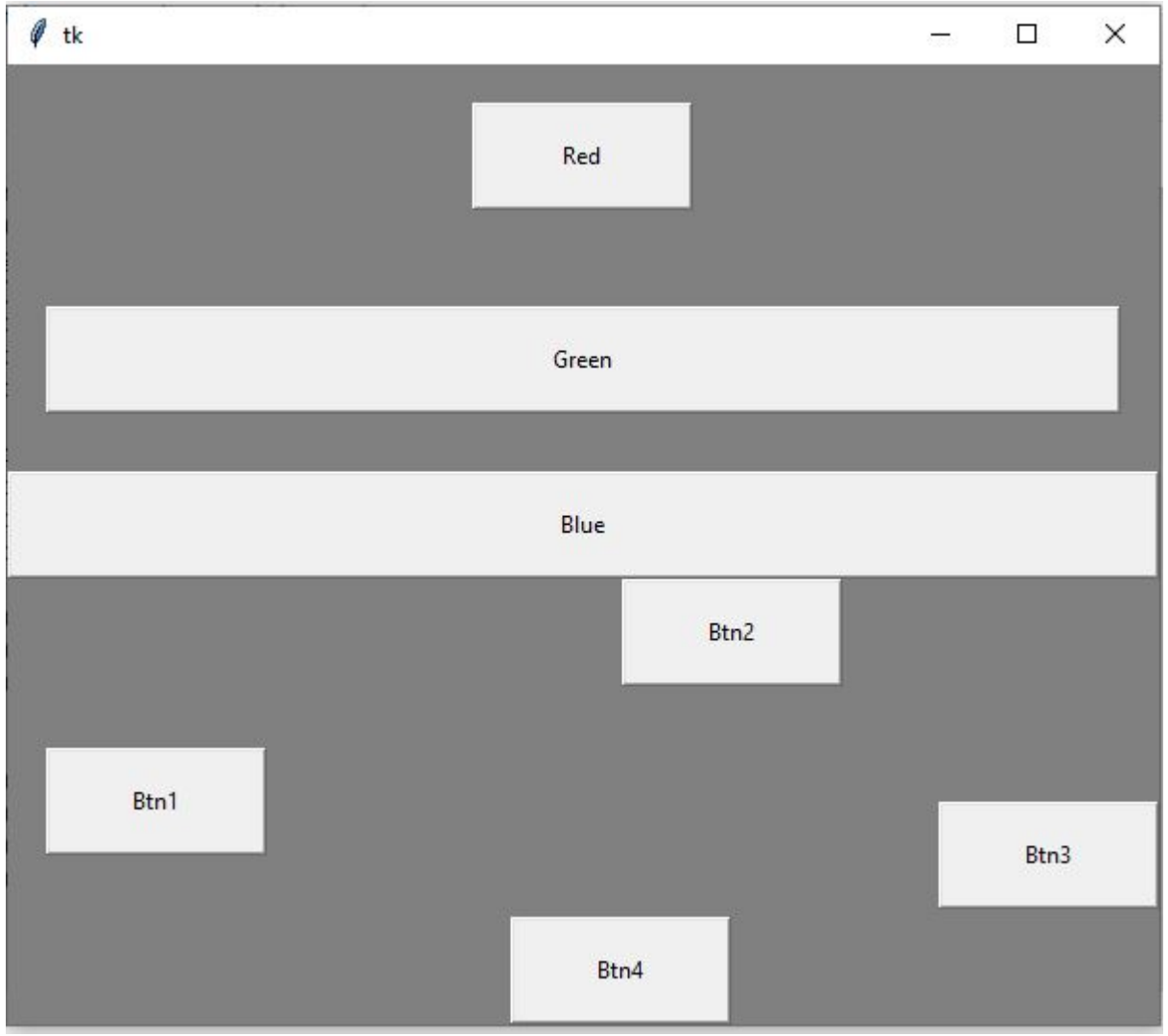
```
        self.b1.pack()
        self.b2.pack()
        self.b3.pack()
    def buttonClick(self,num):
        if num==1:
            self.f["bg"]="red"
        if num==2:
            self.f["bg"]="green"
        if num==3:
            self.f["bg"]="blue"

root=Tk()
mb=MyButton(root)
root.mainloop()
```

- **Widgets-Arranging Widgets in the Frame**
  - When we create widgets or components then we need to arrange then in particular manner
  - Arranging widgets in the frame is called **Layout Management**
  - There are three layout managers as
    - Pack layout manager
    - Grid layout manager
    - Place layout manager

- **Pack layout manager**
  - Pack layout manager uses pack() method.
  - This method associate a widget with its parent component.
  - We can mention position of widget using **fill or side** option
  - Fill option uses value as **X, Y, BOTH** or **NONE** where
    - X represent horizontal occupation,
    - Y represent vertical occupation,
    - BOTH represent widget should occupy in both direction and
    - NONE represent widget should display as it is.
    - Along with fill option we can use **padx** and **pady** options that represents how much space should be left around the component horizontally and vertically
  - Side is used to place the widgets side by side
    - Side can take values as LEFT, RIGHT, TOP or BOTTOM
    - Default value is TOP

- **Pack layout manager**

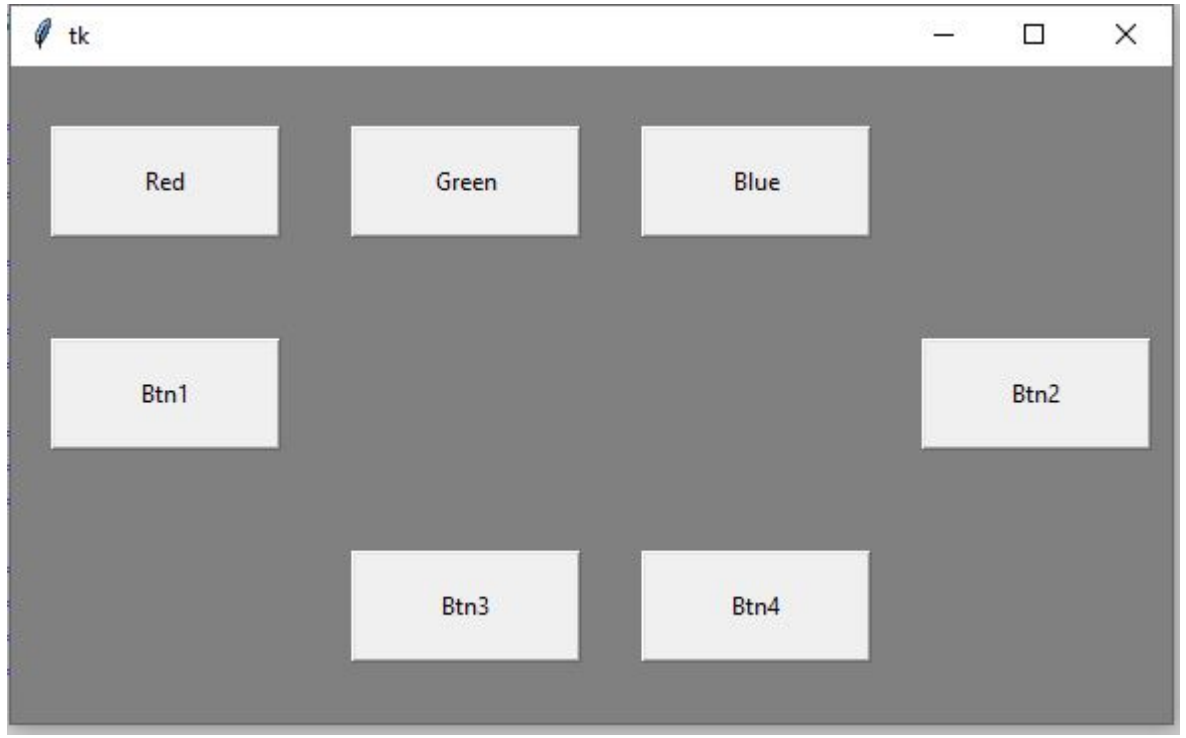
```
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600, bg="gray")
        self.f.propagate(0)
        self.f.pack()
        self.b1=Button(self.f,text="Red", width=15, height=3)
        self.b2=Button(self.f,text="Green", width=15, height=3)
        self.b3=Button(self.f,text="Blue", width=15, height=3)
            self.b4=Button(self.f,text="Btn1", width=15,
height=3)
        self.b5=Button(self.f,text="Btn2", width=15, height=3)
        self.b6=Button(self.f,text="Btn3", width=15, height=3)
        self.b7=Button(self.f,text="Btn4", width=15, height=3)

        self.b1.pack(fill=Y, padx=20, pady=20)
        self.b2.pack(fill=X, padx=20, pady=30)
        self.b3.pack(fill=BOTH)
        self.b4.pack(side=LEFT,  padx=20, pady=20)
        self.b5.pack()
        self.b6.pack(side=RIGHT)
        self.b7.pack(side=BOTTOM)

root=Tk()
mb=MyButton(root)
root.mainloop()
```



- **Grid layout manager**
  - Grid layout manager uses grid() method.
  - This method arrange a widgets in two dimensional table contains row (Horizontal) and column (Vertical).
  - The position of widget are defined by row and column number.
  - The size of the table is determined by the grid layout manager depending on widget size.

- **Grid layout manager**

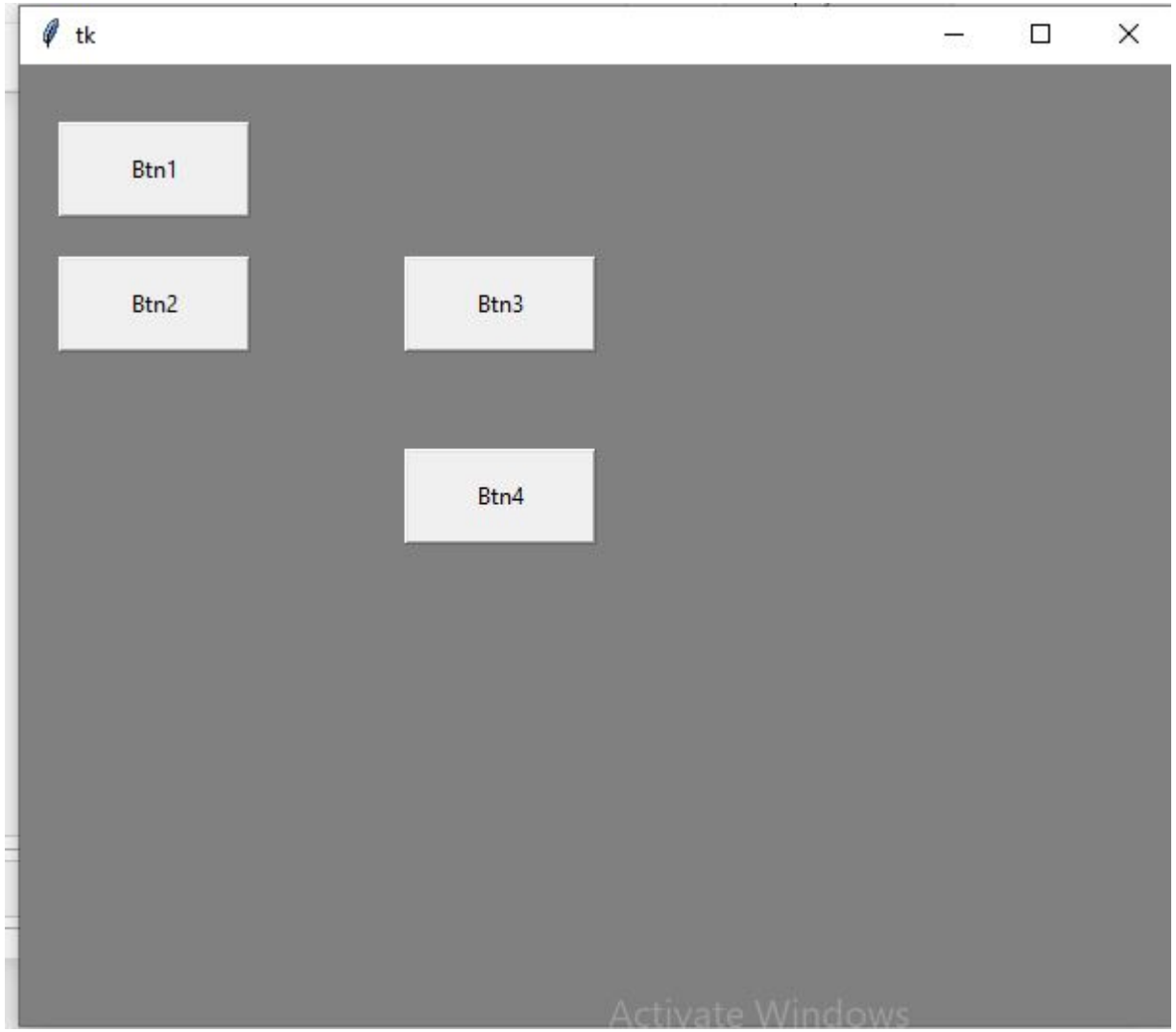
```
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600, bg="gray")
        self.f.propagate(0)
        self.f.pack()
        self.b1=Button(self.f,text="Red", width=15, height=3)
        self.b2=Button(self.f,text="Green", width=15, height=3)
        self.b3=Button(self.f,text="Blue", width=15, height=3)
        self.b4=Button(self.f,text="Btn1", width=15, height=3)
        self.b5=Button(self.f,text="Btn2", width=15, height=3)
        self.b6=Button(self.f,text="Btn3", width=15, height=3)
        self.b7=Button(self.f,text="Btn4", width=15, height=3)

        self.b1.grid(row=0, column=0, padx=20, pady=20)
        self.b2.grid(row=0, column=1, padx=10, pady=10)
        self.b3.grid(row=0, column=2, padx=15, pady=30)
        self.b4.grid(row=1, column=0, padx=20, pady=20)
        self.b5.grid(row=1, column=3, padx=10, pady=10)
        self.b6.grid(row=2, column=1, padx=15, pady=30)
        self.b7.grid(row=2, column=2, padx=15, pady=30)

root=Tk()
mb=MyButton(root)
root.mainloop()
```



- **Place layout manager**
  - Place layout manager uses place() method.
  - This method takes x and y coordinates of widgets along with width and height of the window where widget has to be placed.

- **Place layout manager**

```
from tkinter import *
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600, bg="gray")
        self.f.propagate(0)
        self.f.pack()

        self.b4=Button(self.f,text="Btn1", width=15, height=3)
        self.b5=Button(self.f,text="Btn2", width=15, height=3)
        self.b6=Button(self.f,text="Btn3", width=15, height=3)
        self.b7=Button(self.f,text="Btn4", width=15, height=3)


        self.b4.place(x=20, y=30, width=100, height=50)
        self.b5.place(x=20, y=100, width=100, height=50)
        self.b6.place(x=200, y=100, width=100, height=50)
        self.b7.place(x=200, y=200, width=100, height=50)


root=Tk()
mb=MyButton(root)
root.mainloop()
```



- **Label Widgets-**
  - Label widget is used to display text in the form or container.
  - A label can display one line of text that cannot be modified
  - A label is created as an object of Label class as
  - ***lbl=Label(f, text="Sample Label", width=20, height=3, bg="blue", fg="white", font=('Courier',30,'bold'))***
  - Here f represent the parent frame where we add the label
  - text represent text to display
  - height and width represent height and width of button in pixel
  - bg and fg represent background and font color. Here we directly write color name
  - Font represent font used to label
  - Following are the options used with label widgets

- **Label Widgets-**

Name of the option	Description
anchor	This option is mainly used for controlling the position of text in the provided widget size. The default value is CENTER which is used to align the text in center in the provided space.
bd	This option is used for the border width of the widget. Its default value is 2 pixels.
bitmap	This option is used to set the bitmap equals to the graphical object specified so that now the label can represent the graphics instead of text.
bg	This option is used for the background color of the widget.
cursor	This option is used to specify what type of cursor to show when the mouse is moved over the label. The default of this option is to use the standard cursor.
fg	This option is used to specify the foreground color of the text that is written inside the widget.
font	This option specifies the font type of text inside the label.
height	This option indicates the height of the widget
image	This option indicates the image that is shown as the label.
justify	This option specifies the alignment of multiple lines in the label. The default value is CENTER. Other values are RIGHT, LEFT; you can justify according to your requirement
padx	This option indicates the horizontal padding of the text. The default value of this option is 1.
pady	This option indicates the vertical padding of the text. The default value of this option is 1.

- **Label Widgets-**

Name of the option	Description
relief	This option indicates the type of border. The default value of this option is FLAT
text	This option is set to the string variable and it may contain one or more than one line of text
textvariable	This option is associated with a Tkinter variable that is (StringVar) with a label. If you change the value of this variable then text inside the label gets updated.
underline	This option is used to underline a specific part of the text. The default value of this option =-1(no underline); you can set it to any integer value up to n and counting starts from 0.
width	This option indicates the width of the widget.
wraplength	Rather than having only one line as the label text, you can just break it to any number of lines where each line has the number of characters specified to this option.

- **Label Widgets-**

```
from tkinter import *
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=600, width=500, bg="gray")
        self.f.propagate(0)
        self.f.pack()

        self.b1=Button(self.f,text="Click Me", width=15, height=3,
bg="gray", fg="red", activebackground="green",
activeforeground="white", command=self.buttonClick)
        self.b2=Button(self.f,text="Close", width=15, height=3,
bg="gray", fg="red", activebackground="green",
activeforeground="white", command=self.quit)

        self.b1.grid(row=0, column=1, padx=20, pady=20)
        self.b2.grid(row=0, column=2, padx=10, pady=10)


    def buttonClick(self):
        self.lbl=Label(self.f, text="Welcome to Python", width=20,
height=3, bg="blue", fg="white", font=('Courier',30,'bold'))
        self.lbl.grid(row=2, column=0, padx=10, pady=10)
    def quit(self):
        self.lbl["text"]="Good By"
```

```
root=Tk()
mb=MyButton(root)
root.mainloop()
```

- **Message Widgets-**
  - A message widget is similar to Label but it is used to display multiline text.
  - All the message text will be display in same fort
  - To create message we need to create an object of Message class as
  - ***msg=Message(f, text="This is message Widget and it has more than one line of text", width=300, bg="white", fg="dark goldenrod", font=('Roman',30,'bold'))***
  - Here f represent the parent frame where we add the message widget
  - text represent text to display
  - width represent width of button in pixel
  - bg and fg represent background and font color. Here we directly write color name
  - Font represent font used for message widget
  - Following are the options used with message widget

- **Message Widgets–**

Name of the option	Description
anchor	It is mainly used to decide the exact position of the text within the provided space. The default value of this option is CENTER.
bg, fg	This option denotes the background, font color of the widget.
bd	This option is used to indicate the border width of the widget. The default value of this is 2 pixels.
bitmap	In order to display graphics on the widget, this option will be used. You can set it to any graphical or image object.
cursor	With the help of this option, the mouse pointer will be changed to a specific cursor type like an arrow, dot, etc.
font	This option is used to indicate the font type of the widget text.
height	This option is used to indicate the vertical dimension of the message widget.
image	This option is used to indicate the image on the widget.
justify	This option is used for the justification of the text on the widget. It can be CENTER, LEFT, RIGHT
padx	This option is used for the horizontal padding of the widget.
pady	This option is used for the vertical padding of the widget.

- **Message Widgets–**

Name of the option	Description
relief	This option is used to specify the border type. Its default value is FLAT
underline	This option can be set to an existing number in order to specify that nth letter of the string will be underlined. Its default value is -1 which indicates no underline.
text	If you want to display one or more lines of text in a label widget you need to set this option to a string containing the text. You can use "\n" just in order to enter multiple lines
textvariable	This option is used to control the text represented by the widget. The textvariable can be set to the text that is needed to be shown on the widget.
width	This option is used to indicate the horizontal dimension of the widget in the number of characters and not in pixels.
wraplength	This option is used to wrap the text to the number of lines just by setting this option to the desired number so that each line contains only that number of characters.

- **Message Widgets-**

```
from tkinter import *
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=600, width=800,
bg="gray")
        self.f.propagate(0)
        self.f.pack()


        self.msg=Message(self.f, text="This is message
Widget and it has more than one line of text",
width=300,  bg="white", fg="dark goldenrod",
font=('Roman',30,'bold'))
        self.msg1=Message(self.f, text="Sangola College
Sangola, Kadlas Road, Sangola, Tal-Sangola,
Dist-Solapur, Pin-413307", width=500,  bg="red",
fg="white", font=('Arial',40,'bold italic'))

        self.msg.pack(side=LEFT)
        self.msg1.pack(side=RIGHT)
```

```
root=Tk()
mb=MyButton(root)
root.mainloop()
```

- **Text Widgets-**
  - A Text widget is similar to Label or Message widgets but it has several options and can display multiple line of text in different font and different color.
  - It is possible to insert text into Text widget and also delete or modify it.
  - To create Text widget we need to create an object of Text class as
  - ***txt=Text(f, width=300, height=10, bg="white", fg="dark goldenrod", font=('Roman',30,'bold'), wrap=WORD)***
  - Here f represent the parent frame where we add the Text Widgets
  - Width and height represent width and height of button in pixel
  - bg and fg represent background and font color. Here we directly write color name
  - Font represent font used for Text Widgets
  - wrap option represent where to cut the line. It has 3 possible values CHAR, WORD and NONE
    - CHAR represent cut the line at any character, WORD represent cut the line at last word that fits the line and NONE will not wrap the line so in this case you need scrollbars.

- **Text Widgets-**
  - You can use various styles and attributes with the Text widget.
  - You can also use marks and tabs in the Text widget to locate the specific sections of the text.
  - Media files like images and links can also be inserted in the Text Widget.
  - There are some variety of applications where you need multiline text like sending messages or taking long inputs from users, or to show editable long format text content in application, etc. use cases are fulfilled by this widget.
  - Thus in order to show textual information, we will use the Text widget.

- **Text Widgets-Following are the options used with Text Widgets**

Name of the option	Description
bd	This option represents the border width of the widget.
bg	This option indicates the background color of the widget.
exportselection	This option is used to export the selected text in the selection of the window manager. If you do not want to export the text then you can set the value of this option to 0.
cursor	This option will convert the mouse pointer to the specified cursor type and it can be set to an arrow, dot, etc.
font	This option is used to indicate the font type of the text.
fg	This option indicates the text color of the widget
height	This option indicates the vertical dimension of the widget and it is mainly in the number of lines.
highlightbackground	This option indicates the highlightcolor at the time when the widget isn't under the focus.
higlightthickness	This option is used to indicate the thickness of the highlight. The default value of this option is 1.
highlightcolor	This option indicates the color of the focus highlight when the widget is under the focus.
insertbackground	This option is used to represent the color of the insertion cursor.
padx	This option indicates the horizontal padding of the widget.
pady	This option indicates the vertical padding of the widget.

- **Text Widgets-Following are the options used with Text Widgets**

Name of the option	Description
relief	This option indicates the type of the border of the widget. The default value of this option is SUNKEN.
state	If the value of this option is set to DISABLED then the widget becomes unresponsive to mouse and keyboard
tabs	This option is used to control how the tab character is used for the positioning of the text
width	This option represents the width of the widget and this is in characters.
wrap	To wrap wider lines into multiple lines this option is used. The default value of this option is CHAR which breaks the line which gets too wider at any character
xscrollcommand	If you want to make the Text widget horizontally scrollable, then you can set this option to the set() method of Scrollbar widget
yscrollcommand	If you want to make the Text widget vertically scrollable, then you can set this option to the set() method of Scrollbar widget
spacing1	This option indicates the vertical space to insert above each line of the text.
spacing2	This option is used to specify how much extra vertical space to add between displayed lines of text when a logical line wraps. The default value of this option is 0
spacing3	This option indicates the vertical space to insert below each line of the text.

- **Text Widgets-Following are the options used with Text Widgets**

Name of the option	Description
selectbackground	This option indicates the background color of the selected text.
selectborderwidth	This option indicates the width of the border around the selected text.
insertofftime	This option represents the time amount in Milliseconds and during this time the insertion cursor is off in the blink cycle
insertontime	This option represents the time amount in Milliseconds and during this time the insertion cursor is on in the blink cycle
insertborderwidth	In order to represent the width of the border around the cursor, we use this option. The default value of this option is 0.

- **Text Widgets-Following are the options used with Text Widgets**

Method	Description
index(index)	This method is used to get the specified index.
see(index)	This method returns true or false on the basis that if the string is visible or not at the specified index.
insert(index,string)	This method is used to insert a string at the specified index.
get(startindex,endindex)	This method returns the characters in the specified range
delete(startindex,endindex)	This method deletes the characters in the specified range

- Text Widgets- Methods for Tag Handling
  - Mainly tags are used to configure different areas of the text widget separately.
  - Tag is basically the name given to separate areas of the text.
  - Some Methods for handling tags are given below:
  - tag_config() -
    - To configure the properties of the tag this method will be used.
  - tag_add(tagname, startindex, endindex)
    - This method is mainly used to tag the string that is present at the specified index.
  - tag_delete(tagname)
    - This method is mainly used to delete the specified tag.
  - tag_remove(tagname, startindex, endindex)
    - To remove the tag from the specified range this method is used.

- Text Widgets-Methods for Mark Handling
  - In a given text widget in order to bookmark specified positions between the characters Mark is used. Some methods for the same are given below:
  - index(mark)
    - This method is mainly used to get the index of the mark specified.
  - mark_names()
    - This method is used to get all the names of the mark in the range in the text widget.
  - mark_gravity(mark, gravity)
    - To get the gravity of the given mark this method will be used.
  - mark_set(mark, index)
    - This method is used to inform the new position of the given mark.
  - mark_unset(mark)
    - In order to remove the given mark from the text this method will be used.

- **Text Widgets-**

```
from tkinter import *
class MyButton:
    def __init__(self,root):
        self.f=Frame(root, height=600, width=800, bg="gray")
        self.f.propagate(0)
        self.f.pack()

        self.txt=Text(self.f, width=100, height=10, bg="white", fg="dark
goldenrod", font=('Roman',30,'bold'), wrap=WORD)

        self.l = Label(self.f, text = "Quote for the Day",font=("Courier",14))

        self.Quote = """Success usually comes to those who are too busy
to be looking for it"""



        self.b1 = Button(self.f, text = "Insert", command = self.insert )
        self.b3 = Button(self.f, text = "Highlight Text", command =
self.add_highlighter)
         self.b2 = Button(self.f, text = "Exit", command = self.exit)
```

```
        self.l.pack()
        self.txt.pack()
        self.b1.pack()
        self.b3.pack()
        self.b2.pack()

    def insert(self):
        self.txt.insert(END,self.Quote)
        #self.txt["state"]=DISABLED

    def exit(self):
        self.txt.delete("1.0","end")

    def add_highlighter(self):
        self.txt.tag_add("start", "1.0","1.17")
        self.txt.tag_config("start", background= "black",
foreground= "white")

root=Tk()
mb=MyButton(root)
root.mainloop()
root=Tk()
mb=MyButton(root)
root.mainloop()
```

- Text Widgets- Methods for Tag Handling
  - It is possible to mark some part of the text as a tag and provide different color and font
  - For this purpose first we should specify tag using tag_add() method and we can apply different color and font using tag_config() as
    - *self.txt.tag_add("start", "1.0","1.17")*
    - *self.txt.tag_config("start", background= "black", foreground= "white")*
  - We may also add scrollbar and image in Text widget as shown in next example

- **Text Widgets-**

```
from tkinter import *
class MyText:
    def __init__(self,root):
        self.txt=Text(root, width=30, height=10, bg="white",
fg="dark goldenrod", font=('Roman',30,'bold'), wrap=WORD)
        self.txt.insert(END,"Sangola College Sangola,\nKadlas
Road, Sangola,\nTal-Sangola,\nDist-Solapur,\nPin-413307")
        self.txt.pack(side=LEFT)
        self.logo=PhotoImage(file="F:\\python\\cat.gif")
        self.txt.image_create(END, image=self.logo)
        self.txt.tag_add("start",1.0,1.23)
        self.txt.tag_config("start",
background="red",foreground="white", font=("Lucide
xonsole",15,"bold"))

        self.sv=Scrollbar(root, orient=VERTICAL,
command=self.txt.yview)
        self.txt.configure(yscrollcommand=self.sv.set)
        self.sv.pack(side=RIGHT, fill=Y)
```

```
root=Tk()
mt=MyText(root)
root.mainloop()
```

- **Scrollbar Widgets-**
  - A scrollbar is a widget that is useful to scroll the text in another widget.
  - Ex. Text in the Text widget, Frames or ListBox can be scrolled from top to bottom or left to right
  - There are two types of scrollbars- **Horizontal and Vertical**
  - To create horizontal scrollbar we have to create object of Scrollbar class as
  - ***hs=Scrollbar(root, orient=HORIZONTAL, command=t.xview)***
  - Here hs represents the scrollbar object created as a child of root window
  - orient represent scrollbar direction. Possible values are HORIZONTAL and VERTICAL
  - The option command represent the method that is to be executed. Here xview is executed on the object t (Ex. Text).
  - To create horizontal scrollbar we have to create object of Scrollbar class as
  - ***vs=Scrollbar(root, orient=VERTICAL, command=t.yview)***

- **Scrollbar Widgets-**
  - After creating scrollbar it should be attached to widgets like Text or Listbox widgets as
  - ***t.configure(xscrollcommand=hs.set)***
  - Here t indicates Text widgets
  - xscrollcommand calls set() method of horizontal scroll bar and
  - yscrollcommand calls set() method of vertical scroll bar
  - Finally the scrollbar should be attached root window using pack() or grid() method as
  - ***hs.pack(side=BOTTOM, fill=X)***
  - Here we attaching horizontal scroll bar at the bottom of widget and it spreads across X axis.
  - Similarly we can attach vertical scrollbar as
  - v***s.pack(side=RIGHT, fill=Y)***

- **Scrollbar Widgets– Important Options available for Scrolbar**

Name of the Option	Description
activebackground	This option represents the background color of the widget when it is under focus.
bg	This option represents the background color of the widget
bd	This option represents the border size of the widget. The default value is 2 pixels.
cursor	With the help of this option, the mouse pointer will be changed to a specific cursor type and it can be an arrow, dot, etc.
command	This option will be set to the procedure associated which is called every time the scrollbar is moved.
elementborderwidth	This option mainly represents the border width around the arrowheads and the slider. The default value of this option is –1.
highlightthickness	This option represents the thickness of the focus highlights
highlightbackground	This option indicates the highlight color when the widget is not under the focus
highlightcolor	This option indicates the highlight color when the widget is under the focus
jump	This option is used to control the behavior of the scroll jump. If this option is set to 1, then the callback is called at the time when the user releases the mouse button.

- **Scrollbar Widgets-**

Name of the Option	Description
orient	This option can be set to either horizontal or vertical depending upon the orientation of the scrollbar.
width	This option represents the width of the scrollbar.
troughcolor	This option is used to set the color for the trough
takefocus	By default, you can tab the focus through this widget. If you don't want this behavior you can set this option to 0.
repeatdelay	This option is mainly used to tell the duration up to which the button is to be pressed before the slider starts moving in that direction repeatedly. its default value is 300 ms
repeatinterval	The default value of this option is 100

- **Scrollbar Widgets-**

```
from tkinter import *
class MyText:
    def __init__(self,root):
        self.txt=Text(root, width=70, height=10,
bg="white", fg="dark goldenrod",
font=('Roman',30,'bold'), wrap=NONE)
        for i in range(50):
            self.txt.insert(END,"This is sample text")

        self.txt.pack(side=TOP, fill=Y)

        self.sh=Scrollbar(root, orient=HORIZONTAL,
command=self.txt.xview)
        self.txt.configure(xscrollcommand=self.sh.set)
        self.sh.pack(side=BOTTOM, fill=X)

        root=Tk()
mt=MyText(root)
root.mainloop()
```

Horizontal Scrollbar

```
from tkinter import *
class MyText:
    def __init__(self,root):
        self.sv=Scrollbar(root, orient=VERTICAL)
        self.sv.pack(side=RIGHT, fill=Y)

        self.sh=Scrollbar(root, orient=HORIZONTAL)
        self.sh.pack(side=BOTTOM, fill=X)

        self.txt=Text(root, width=10, height=10, bg="white", fg="dark
goldenrod", font=('Roman',30,'bold'),
wrap=NONE,xscrollcommand=self.sh.set, yscrollcommand=self.sv.set)
        for i in range(50):
            self.txt.insert(END,"This is sample text")

        self.txt.pack(side=TOP, fill=BOTH)
        self.sh.config(command=self.txt.xview)
        self.sv.config(command=self.txt.yview)

root=Tk()
mt=MyText(root)
root.mainloop()
```

Both Side Scrollbar

- **Checkbox/Checkbutton Widgets-**
  - It is used to select one or more options from available group option.
  - When a check button is selected, a tick mark is displayed on the button
  - We can create check buttons using object of Checkbutton class as
  - ***chk1=Checkbutton (f, bg="blue", fg="white", font=('Courier',30,'bold'), text="Python", variable=var, command=display)***
  - Here chk1 represents the checkbutton object of Checkbutton class
  - text represent the text to be displayed after the checkbutton
  - variable represent an object of IntVar() class (***var=IntVar()***). It is used to now the state of checkbutton whether checked or not. Here vale of var is 1 it means checkbutton is selected otherwise 0. we can retrieve value of var by using get() as ***x=var.get()***
  - command represent the method to be called when the user clicks checkbutton

- **Checkbox/Checkbutton Widgets-**

Option name	Description
activebackground	This option indicates the background color of the checkbutton at the time when the checkbutton is under the cursor.
bd	This option indicates the size of the border around the corner. The default size is 2 pixels.
bg,fg	This option is used to represent the background and forground color of the checkbutton.
bitmap	This option is mainly used to display the image on the button.
command	The command option is used to set the function call which is scheduled at the time when the state of checkbutton is changed.
activeforeground	This option mainly represents the foreground color of the button when the checkbutton is under the cursor.
font	This option indicates the font of the checkbutton.
height, width	This option indicates the height and width of the button. This height indicates the number of text lines in the case of text lines and it indicates the number of pixels in the case of images. the default value is 1. This option specifies the width of the checkbutton. For textual buttons, It exists as a number of letters or for image buttons it indicates the pixels.
image	This option indicates the image representing the checkbutton.
cursor	This option helps in changing the mouse pointer to the cursor name when it is over the checkbutton.

- **Checkbox/Checkbutton Widgets-**

Option name	Description
disableforeground	This option is the color which is used to indicate the text of a disabled checkbutton.
higlightcolor	This option indicates the highlight color when there is a focus on the checkbutton
justify	This option is used to indicate the way by which the multiple text lines are represented. For left justification, it is set to LEFT and it is set to RIGHT for the right justification, and CENTER for the center justification.
padx	This option indicates the padding of the checkbutton in the horizontal direction.
pady	This option indicates the padding of the checkbutton in the vertical direction.
underline	This option is used to underline the text of the checkbutton.
Wraplength	In the case If this option is set to an integer number, then the text will be broken into the number of pieces.
variable	This option is mainly used to represents the associated variable that is used to track the state of the checkbutton
offvalue	The associated control variable of checkbutton is set to 0 by default if the button is (off). you can also change the state of an unchecked variable to some other one.
onvalue	The associated control variable of checkbutton will be set to 1 when it is set (on). Any alternate value will be supplied for the on state by setting onvalue to that value.

- **Checkbox/Checkbutton Widgets-**

Option name	Description
text	This option is used to indicate the label just next to the checkbutton. For multiple lines use "\n".
state	This option mainly used to represent the state of the checkbutton. Its default value= normal. It can be changed to DISABLED to make the checkbutton unresponsive. The value of this button is ACTIVE when checkbutton is under focus
selectcolor	This option indicates the color of the checkbutton when it is set. Its default value is Red.
selectimage	This option indicates the image on the checkbutton when it is set.

- **Checkbox/Checkbutton Widgets- Methods**

Method Name	Description
invoke()	This method in checkbutton widget is used to invoke the method associated with the checkbutton.
select()	This method in the checkbutton widget is called to turn on the checkbutton.
deselect()	This method in the checkbutton widget is called to turn off the checkbutton.
toggle()	This method in the checkbutton widget is used to toggle between the different Checkbuttons.
flash()	This method in the checkbutton widget is used to flashed between active and normal colors.

- **Checkbox/Checkbutton Widgets-**

```
from tkinter import *
class MyCheckbutton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600)
        self.f.propagate(0)
        self.f.pack()

        self.java=IntVar()
        self.python=IntVar()
        self.cpp=IntVar()
        self.csharp=IntVar()
        fnt=("Georgia",20,"underline")
        self.chkjava=Checkbutton(self.f, bg="yellow", fg="green",
font=fnt, text="Java", variable=self.java, command=self.display)
        self.chkpython=Checkbutton(self.f, bg="yellow", fg="green",
font=fnt, text="python", variable=self.python, command=self.display)
        self.chkcpp=Checkbutton(self.f, bg="yellow", fg="green",
font=fnt, text="cpp", variable=self.cpp, command=self.display)
        self.chkcsharp=Checkbutton(self.f, bg="yellow", fg="green",
font=fnt, text="csharp", variable=self.csharp, command=self.display)
```

```
self.chkjava.place(x=50,y=100)
        self.chkpython.place(x=50,y=150)
        self.chkcpp.place(x=50,y=200)
        self.chkcsharp.place(x=50,y=250)

def display(self):
        j=self.java.get()
        p=self.python.get()
        cp=self.cpp.get()
        cs=self.csharp.get()

        status=""
        if j==1:
            status+="Java"
        if p==1:
            status+="Python"
        if cp==1:
            status+="CPP"
        if cs==1:
            status+="C Sharp"

        self.lbl=Label(self.f,text=status,bg="red", fg="white", font=('Roman',20,"bold"))
        self.lbl.place(x=100,y=50)

root=Tk()
mb=MyCheckbutton(root)
root.mainloop()
```

- **Radiobutton Widgets-**
  - It is similar to checkbutton but it is useful to select only one option from available group option.
  - We can create check buttons using object of Checkbutton class as
  - ***rdo1=Radiobutton (f, bg="blue", fg="white", font=('Courier',30,'bold'), text="Male", variable=var, value=1, command=display)***
  - Here rdo1 represents the radiobutton object of Radiobutton class
  - text represent the text to be displayed after the checkbutton
  - Value represent a value that is set to the object i.e. (1 is true)
  - variable represent an object of IntVar() class (***var=IntVar()***). It is used to now the state of checkbutton whether checked or not. Here vale of var is 1 it means checkbutton is selected otherwise 0. we can retrieve value of var by using get() as ***x=var.get()***
  - command represent the method to be called when the user clicks checkbutton

- **Radiobutton Widgets–**

Name of the option	Description
anchor	This option is used to represent the exact position of the text within the widget, in the case of the widget contains more space than the requirement of the text. The default value of this option is CENTER.
bg	This option represents the background color of the widget.
activebackground	This option represents the background color of the widget when it is under focus.
activeforeground	This option represents the font color of the widget when it is under focus.
borderwidth	This option is used to represent the size of the border.
bitmap	If you want to display graphics on the widget then you can set this widget to any graphical or image object.
command	This option is used to set the procedure which must be called every time when the state of the radiobutton is changed.
cursor	This option will convert the mouse pointer to the specified cursor type and it can be set to an arrow, dot, etc.
font	This option is used to represent the font type of the text of the widget.
fg	This option is used to represent the foreground color of the text of the widget.
height	This option indicates the vertical dimension of the widget
width	This option indicates the horizontal dimension of the widget and it is represented as the number of characters.

- **Radiobutton Widgets–**

Name of the option	Description
padx	This option represents the horizontal padding of the widget.
pady	This option represents the vertical padding of the widget
highlightcolor	This option is used to represent the color of the focus highlight when the widget is under the focus
highlightbackground	This option is used to represent the color of the focus highlight when the widget is not under the focus.
image	If you want to display an image on the widget then this option will be set to an image rather than the text
justify	This option is used to represent the justification of the multiline text. The default value is CENTER. Other values are LEFT, RIGHT.
relief	This option is used to represent the type of border. The default value is FLAT.
selectcolor	This option indicates the color of the radiobutton when it is selected
selectimage	This option indicates the image to be displayed on the radiobutton when it is selected
state	This option is used to represent the state of the radio button. The default state of the Radiobutton is NORMAL. You can also set the state to DISABLED in order to make the radiobutton unresponsive.
text	This option indicates the text to be displayed on the radiobutton.

- **Radiobutton Widgets–**

Name of the option	Description
text	This option indicates the text to be displayed on the radiobutton.
textvariable	This option is used to control the text represented by the widget. The textvariable can be set to the text that is needed to be shown on the widget.
underline	This option can be set to an existing number in order to specify that nth letter of the string will be underlined. Its default value is -1 which indicates no underline
variable	This option is also known as the control variable which is used to keep the track of user's choices. Thus this variable is shared among all radiobuttons.
value	This option of each radiobutton is assigned to the control variable when it is turned on by the user.
wraplength	This option is used to wrap the text to the number of lines just by setting this option to the desired number so that each line contains only that number of characters.

- **Radiobutton Widgets– Methods**

Method Name	Description
deselect()	This method is used to deselect or turns off the radio button
select()	This method is used to select the radio button
invoke()	This method is generally used to call a function when the state of radio button gets changed.
flash()	This method is generally used to flash the radio button between its normal and active colors many times.

- **Radiobutton Widgets-**

```
from tkinter import *
class MyRadiobutton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600)
        self.f.propagate(0)
        self.f.pack()

        self.cls=IntVar()



        self.lbl1=Label(self.f,text="Choose Class",bg="Blue", fg="white",
font=('Roman',30,"bold"))
        self.lbl1.place(x=30,y=10)
        fnt=("Georgia",20,"underline")
        self.rdoecs1=Radiobutton(self.f, bg="yellow", fg="green",
font=fnt, text="ECS-I", variable=self.cls, command=self.display,
value=1)
        self.rdoecs2=Radiobutton(self.f, bg="yellow", fg="green",
font=fnt, text="ECS-II", variable=self.cls, command=self.display,
value=2)
        self.rdoecs3=Radiobutton(self.f, bg="yellow", fg="green",
font=fnt, text="ECS-III", variable=self.cls, command=self.display,
value=3)

        self.rdoecs1.place(x=50,y=100)
        self.rdoecs2.place(x=50,y=150)
```

```
def display(self):

        x=self.cls.get()



        status=""
        if x==1:
            status="ECS-I"
        if x==2:
            status="ECS-II"
        if x==3:
            status="ECS-III"

        self.lbl=Label(self.f,text=status,bg="red", fg="white", font=('Roman',20,"bold"))
        self.lbl.place(x=100,y=300)

root=Tk()
mb=MyRadiobutton(root)
root.mainloop()
```

- **Radiobutton Widgets-**

```
from tkinter import *
class MyRadiobutton:
    def __init__(self,root):
        self.f=Frame(root, height=500, width=600)
        self.f.propagate(0)
        self.f.pack()

        self.cls=IntVar()

        options={"Option1":1,"Option2":2,"Option3":3,"Option4":4}

        for (txt,val) in options.items():
            Radiobutton(self.f,text=txt,variable=self.cls,value=val).pack()

root=Tk()
mb=MyRadiobutton(root)
root.mainloop()
```

- **Entry Widgets-**
  - Entry widget is useful to create rectangular box that can be used to enter or display one line of text.
  - It is similar to TextBox control in C# or ASP.Net
  - We can create entry widget using object of Entry class as
  - ***e1=Entry (f, bg="blue", fg="white", font=('Courier',30,'bold'), width=30, show="*")***
  - Here e1 represents the object of Entry class
  - show represent character that replaces the originally typed character (in this case it is *)
  - After typing the text in Entry widget, user press enter button so such event should be linked with Entry widget with bind() method as
  - ***e1.bind("<Return>",self.display)***
  - Where user press enter/return button the event is passed to display method. So we catch this event through ***event*** Keyword in display method as
  - ***def display(self,event):***
  - the event argument never used inside the method.

- **Entry Widgets- Options**

Option Name	Description
bg	This option is used for the background color of the widget.
bd	This option is used for the width of the border in pixels. Its default value is 2 pixels.
cursor	This option helps in changing the mouse pointer to the cursor type and set it to the arrow, dot, etc.
exportselection	It is important to note that By Default, the text that is written inside the entry box will get automatically copied to the clipboard. If you do not want to copy the text then set the value of exportselection to 0.
fg	This option is used to indicate the color of the text.
font	This option is used to represent the font type of the text
highlightbackground	This option is used to represent the color to display in the traversal highlight region when the widget does not have the input focus.
highlightcolor	This option is used to represent the color to use for the traversal highlight rectangle which is drawn around the widget when the widget has an input focus.
justify	This option is used to specify how the text is organized in the case if the text contains multiple lines.
relief	This option is used to indicate the type of border. The default value of this option is FLAT. It has more values like GROOVE, RAISED,RIGID.

- **Entry Widgets- Options**

Option Name	Description
selectbackground	This option is used to indicate the background color of the selected text.
selectforeground	It is used to set the font of the selected task.
selectborderwidth	This option indicates the width of the border to display around the selected task
width	This option indicates the width of the image or width of text to display.
textvariable	With the help of this option, you will be able to retrieve the current text from your entry widget, you need to set this option to an instance of the StringVar class.
show	This option is used to show the entry text of some other type instead of the string. For example, we type the password using stars (*).
xscrollcommand	You can link the entry widget to the horizontal scrollbar if you want the user to enter more text rather then the actual width of the widget.
insertbackground	This option mainly represents the color to use as a background in the area covered by the insertion cursor. and thus this color will normally override the normal background for the widget.

- **Entry Widgets- Methods**

Method Name	Description
delete(first, last=None)	This method is used to delete the specified characters inside the widget.
get()	This method is used to get the entry widget's current text as a string.
icursor(index)	This method is used to set the insertion cursor just before the character at the specified index.
index(index)	This method is used to place the cursor to the left of the character written at the specified index.
select_clear()	This method is used to clear the selection in the case if some selection has been done.
select_present()	If there is a presence of some selection then this method will return true otherwise, it will return false.
insert(index, s)	This method is mainly used to insert the specified string(s) before the character placed at the specified index

- **Entry Widgets- Methods**

Method Name	Description
select_adjust(index)	This method mainly includes the selection of the character present at the specified index
select_form(index)	This method mainly sets the anchor index position to the character specified by the index.
select_range(start, end)	This method is used to select the characters to exist between the specified range
select_to(index)	This method mainly selects all the characters from the beginning to the specified index
xview(index)	This method is used to link the entry widget to a horizontal scrollbar
xview_scroll(number, what)	This method is mainly used to make the entry widget scrollable horizontally

- **Entry Widgets-**

```
from tkinter import *
class MyEntry:
    def __init__(self,root):
        self.f=Frame(root,height=600, width=600, bg="gray")
        self.f.propagate(0)
        self.f.pack()

        fnt=("Georgia",20,"underline")
        self.lblUserName=Label(self.f,text="User
Name:-",bg="yellow",font=fnt)

self.lblPassword=Label(self.f,text="Password:-",bg="yellow
",font=fnt)

self.etrUserName=Entry(self.f,width=20,bg="yellow",fg="bl
ue",font=fnt)

self.etrPassword=Entry(self.f,width=20,bg="yellow",fg="blu
e",font=fnt,show="*")

        self.etrUserName.bind("<Return>",self.display)
        self.etrPassword.bind("<Return>",self.display)
```

```
self.lblUserName.place(x=50,y=100)
        self.lblPassword.place(x=50,y=200)
        self.etrUserName.place(x=250,y=100)
        self.etrPassword.place(x=250,y=200)

    def display(self,event):
        str1=self.etrUserName.get()
        str2=self.etrPassword.get()

        self.lbl1=Label(self.f,text="User Name is -"+str1,bg="black",
fg="dark
goldenrod",font=("Roman",20,"underline")).place(x=100,y=300)
        self.lbl2=Label(self.f,text="Password is -"+str2, bg="black",
fg="dark
goldenrod",font=("Roman",20,"underline")).place(x=100,y=400)
root=Tk()
me=MyEntry(root)
root.mainloop()
```

- **Entry Widgets-**

```
from tkinter import *
class MyName:
    def __init__(self,root):
        self.f=Frame(root,width=700,height=500,bg="gray")
        self.f.propagate(0)
        self.f.pack()

        fnt=("Georgia",15,"bold")
        self.entryText = StringVar()

        self.lblName=Label(self.f,text="Enter Your
Name:-",font=fnt, bg="gray")


        self.etrName=Entry(self.f,width=20, bg="dark gray",
fg="maroon",font=fnt)
        self.etrDisplay=Entry(self.f,width=20, bg="dark gray",
fg="maroon",font=fnt, textvariable=self.entryText)
        self.etrName.bind("<Return>",self.display)
```

```
self.lblName.place(x=35,y=20)
        self.etrDisplay.place(x=175,y=100)
        self.etrName.place(x=270,y=25)

    def display(self,event):
        self.entryText.set( "Hello "+self.etrName.get() )

root=Tk()
calc=MyName(root)
root.mainloop()
```

- **Spinbox Widgets-**
  - Spin box widget is allow user to select value from a given set of values and it may be number or fixed set of strings.
  - User can also edit the value being displayed in the spin box just like Entry widget.
  - It loo like combo box control in C#.
  - We can create spinbox widget using object of **Spinbox** class as
  - ***s1= Spinbox (f, from_=5, to=15, textvariable=var1, width=15, bg="blue", fg="white", font=('Courier',30,'bold))***
  - Here s1 represents the object of Spinbox class
  - from_ and to represent starting and ending value in the spinbox
  - textvariable represent the control variable i.e. var1 is object of IntVar() or StringVar() class.
  - var1 is control variable that receives the displayed value of spinbox.
  - We also create spinbox of strings by specifying the strings as tuple using value option as
  - ***s2= Spinbox (f, values=('ECS-I', ECS-II', ECS-III', BCA-I') textvariable=var2, width=15, bg="blue", fg="white", font=('Courier',30,'bold))***

- **Spinbox Widgets-**
  - Here in s2, var2 is control variable which is object of StringVar() class.
  - To retrieve values from control variable we can use get() method as
  - *a=var1.get() and b=var2.get()*

- **Spinbox Widgets–**

Name of the Option	Description
bg	This option is used for the background color of the widget.
bd	This option is used for the border width of the widget
command	This option is used to indicate the associated function with the widget which is called every time the state of the widget is changed.
cursor	With the help of this option, your mouse pointer type can be changed to the cursor type that is assigned to this option.
activebackground	This option indicates the background color of the widget when it is under the focus
disabledbackground	This option is used to indicate the background color of the widget when it is disabled.
disabledforeground	This option is used to indicate the foreground color of the widget when it is disabled.
font	This option specifies the font type of text inside the widget.
fg	This option specifies the foreground color of the widget.
format	This option is mainly used for the format string. There is no default value of this option.
from_	This option is used to indicate the starting range of the widget
justify	This option specifies the alignment of multiple lines in the label. The default value is LEFT. Other values are RIGHT and CENTER.
relief	This option indicates the type of border. The default value of this option is SUNKEN.

- **Spinbox Widgets-**

```
from tkinter import *
class MySpinbox:
    def __init__(self,root):
        self.f=Frame(root,width=700,height=600,bg="gray")
        self.f.propagate(0)
        self.f.pack()

        fnt=("Georgia",15,"bold")
        self.lbl1=Label(self.f,text="Select Year:-",font=fnt,
bg="gray")
        self.s1=Spinbox(self.f,from_="1970", to="2000",
width=15,font=fnt,)

        self.lbl2=Label(self.f,text="Select City:-",font=fnt,
bg="gray")
self.s2=Spinbox(self.f,values=("Pune","Sangli","Satara","So
lapur"), width=15,font=fnt,)

        self.btn=Button(self.f,text="Display Selected
Value",width=30,height=3,font=("Arial",15),command=self.
display)
```

```
 self.lbl1.place(x=50,y=50)
        self.s1.place(x=200,y=50)
        self.lbl2.place(x=50,y=150)
        self.s2.place(x=200,y=150)
        self.btn.place(x=100,y=270)

    def display(self):
        sp1=self.s1.get()
        sp2=self.s2.get()
        str1="Selected Year is {0} and Selected City is
{1}".format(sp1,sp2)

self.lbl3=Label(self.f,text=str1,bg="green",fg="white",font=
("Georgia",15,"bold"))
        self.lbl3.place(x=70,y=400)

root=Tk()
ms=MySpinbox(root)
root.mainloop()
```

- **Spinbox Widgets-**

Name of the Option	Description
state	This option is used to represent the state of the widget. The default value of this option is NORMAL. Other values are "DISABLED", "read-only", etc.
validate	This option is used to control how to validate the value of the widget
to	This option represents the maximum limit of the widget value. The other value is specified by the from_ option
repeatdelay	This option is mainly used to control the autorepeat button. The value here is in milliseconds.
repeatinterval	This option is similar to repeatdelay option. The value here is also given in milliseconds.
validatecommand	This option is associated with the function callback that is used for the validation of the content of the widget.
xscrollcommand	This option is mainly used with the set() method of the scrollbar widget to make this widget horizontally scrollable
wrap	This option is mainly used to wrap-up the up and down button of the Spinbox
width	This option indicates the width of the widget.
vcmd	This option is similar to validatecommand.
values	This option represents the tuple which contains the values for the widget
textvariable	It is a control variable that is used to control the text of the widget

- **Listbox Widgets-**
  - Listbox is useful to display list of items in the box ant then user can select one or more items.
  - It look like list box control in C#.
  - We can create Listbox widget using object of **Listbox** class as
  - ***l1= Listbox (f, width=15, height=8, bg="blue", fg="white", font=('Courier',30,'bold), activestyle="underline, selectmode=MULTIPLE)***
  - Here l1 represents the object of Listbox class
  - Activestyle represent appearance of the selected item. Possible values are ***underline, dotbox or none.*** Default value is ***underline***
  - selectmode has following possible values
    - ***BROWSE***
    - ***SINGLE***
    - ***MULTIPLE***
    - ***EXTENDED***

- **Listbox Widgets-**
  - selectmode has following possible values
    - ***BROWSE***
      - If we click an item and then drag to different item, selection will follow the mouse. This is default value for selectmode option
    - ***SINGLE***
      - We can select only one item from all available list of items
    - ***MULTIPLE***
      - We can select one or more number of items from all available list of items, If we click repeated on same item it will deselect.
    - ***EXTENDED***
      - We can select any adjacent group of items at once by clicking on the first item and dragging to last item.

- **Listbox Widgets-**
  - At runtime we may add items in list using ***insert()*** method as
    - ***l1.insert(0,"blue")***
    - ***l1.insert(1,"red")***
  - To bind ListboxSelect event with specific method, we can use bind() method as
    - ***l1.bind("<<ListboxSelect>>",On_select)***
    - Here when user select any item from listbox l1, the method On_select will execute.
    - Here function definition is as
    - ***def On_select(event):***
    - Here event parameter is used to catch ListboxSelect event.
  - To retrieve the index or position of selected item, we can use ***curselection()*** method.
  - If we want name of selected item then ***get()*** method is used

- **Listbox Widgets-**

Name of Option	Description
bg	This option indicates the background color of the widget.
bd	This option is used to represent the size of the border. The default value is 2 pixels.
cursor	With the help of this option, the mouse pointer will look like the cursor type like dot, arrow, etc.
font	This option indicates the font type of the Listbox items.
fg	This option indicates the color of the text.
height	This option is used to represents the count of the lines shown in the Listbox. The default value of this option is 10.
highlightcolor	This option is used to indicate the color of the Listbox items when the widget is under focus.
highlightthickness	This option is used to indicate the thickness of the highlight.

- **Listbox Widgets–**

Name of Option	Description
relief	This option indicates the type of border. The default value is SUNKEN.
selectbackground	This option is used to indicate the background color that is used to display the selected text.
selectmode	This option is used to determine the number of items that can be selected from the list. It can set to BROWSE, SINGLE, MULTIPLE, EXTENDED.
width	This option is used to represent the width of the widget in characters.
xscrollcommand	This option is used to let the user scroll the Listbox horizontally.
yscrollcommand	This option is used to let the user scroll the Listbox vertically.

- **Listbox Widgets-Methods**

Method	Description
activate(index)	This method is mainly used to select the lines at the specified index.
curselection()	This method is used to return a tuple containing the line numbers of the selected element or elements, counting from 0. If nothing is selected, return an empty tuple.
delete(first, last = None)	This method is used to delete the lines which exist in the given range.
get(first, last = None)	This method is used to get the list of items that exist in the given range.
index(i)	This method is used to place the line with the specified index at the top of the widget.
insert(index, *elements)	This method is used to insert the new lines with the specified number of elements before the specified index.
nearest(y)	This method is used to return the index of the nearest line to the y coordinate of the Listbox widget.

• Listbox Widgets-Methods

Method	Description
see(index)	This method is used to adjust the position of the Listbox to make the lines specified by the index visible.
size()	This method returns the number of lines that are present in the Listbox widget.
xview()	This method is used to make the widget horizontally scrollable.
xview_moveto(fraction)	This method is used to make the Listbox horizontally scrollable by the fraction of the width of the longest line present in the Listbox.
xview_scroll(number, what)	This method is used to make the listbox horizontally scrollable by the number of characters specified.
yview()	This method allows the Listbox to be vertically scrollable.
yview_moveto(fraction)	This method is used to make the listbox vertically scrollable by the fraction of the width of the longest line present in the Listbox.
yview_scroll (number, what)	This method is used to make the listbox vertically scrollable by the number of characters specified.

- **Listbox Widgets-**

```
class MyListbox:
    def __init__(self,root):
        self.f=Frame(root, width=700, height=500, bg="light
gray")
        self.f.propagate(0)
        self.f.pack()
        fnt=("Georgia",15,"bold")
        self.lbl=Label(self.f,font=fnt, bg="green", fg="white",
text="List of Programming Languages")
        self.lbl.place(x=50,y=50)

        self.lbl1=Label(self.f,font=fnt, bg="blue", fg="red",
text="Select one or More languages")
        self.lbl1.place(x=50,y=100)

        self.lbl2=Label(self.f,font=fnt, bg="dark cyan",
fg="white", text="Your Selections are")
        self.lbl2.place(x=300,y=160)
        self.txt=Text(self.f,font=fnt,bg="maroon",
fg="white",width=20, height=4)
        self.txt.place(x=300,y=200)
```

```
  lst=["C", "C++", "Java", "C#", "Python", "PHP"]
        self.l1=Listbox(self.f,font=fnt,selectmode=MULTIPLE, width=15,
height=4, activestyle="underline")
        self.l1.place(x=70,y=200)

        for itm in lst:
            self.l1.insert(END,itm)

        self.l1.bind("<<ListboxSelect>>",self.on_select)

    def on_select(self,event):
        self.sel_itm=[]
        indexes=self.l1.curselection()
        for i in indexes:
            self.sel_itm.append(self.l1.get(i))

        self.txt.delete(0.0,END)
        self.txt.insert(0.0,self.sel_itm)

root=Tk()
ml=MyListbox(root)
root.mainloop()
```

- **LabelFrame Widgets-**
  - The LabelFrame widget is mainly used to draw borders around the child widgets.
  - It look like Groupbox control in Asp.Net.
  - This widget is a bordered container widget and is used to group the related widgets in a Tkinter application to provide a better user experience to the user.
  - One can also add a title for the LabelFrame widget.
  - The LabelFrame widget is simply a variant of the Frame widget and it has all the features of a frame.
  - We can create LabelFrame widget using object of **LabelFrame** class as
  - ***lf= LabelFrame(f, width=15, height=8, bg="blue", fg="white", font=('Courier',30,'bold))***

- **LabelFrame Widgets-**

Name of the Option	Description
height	This option is used to represent the height of the widget.
width	This option is used to represent the width of the frame.
text	This option represents the string containing the text of the Label.
relief	This option represents the style of the border.The default value of this option is GROOVE
padx	This option represents the horizontal padding of the widget
pady	This option represents the vertical padding of the widget
font	This option represents the font type of the text of the widget
highlighthickness	This option represents the width of the focus highlight border
highlightbackground	This option indicates the color of the focus highlight border at the time when the widget doesn't have the focus
highlightcolor	This option indicates the color of the focus highlight when the widget is under the focus
bg	This option indicates the background color of the widget
bd	This option is used to represent the size of the border around the indicator.The default value of this option is 2 pixels.

- **LabelFrame Widgets-**

Name of the Option	Description
Class	The default value of this option is LabelFrame.
colormap	This option is mainly used to specify which colomap to be used for this widget.With the help of this option, we can reuse the colormap of another window on this widget.The colormap means 256 colors that are used to form the graphics
container	The LabelFrame becomes the container widget if we will set the value of this option to true.The default value of this option is false
cursor	This option will convert the mouse pointer to the specified cursor type and it can be set to an arrow, dot, etc
fg	This option is used to indicate the foreground color of the widget
labelAnchor	This option represents the exact position of the text inside the widget. The default value of this option is NW(north-west)
labelwidget	This option indicates the widget to be used for the label. Also,the frame uses the text for the label if no value specified

- **LabelFrame Widgets-**

```
class MyLabelFrame:
    def __init__(self,root):
        self.f=Frame(root,width=500, height=400, bg="light gray")
        self.f.propagate(0)
        self.f.pack()

        self.labelframe1 = LabelFrame(self.f, text="Happy Thoughts!!!")
        self.labelframe1.pack(fill="both", expand="yes")
        self.toplabel = Label(self.labelframe1, text="You can put your happy thoughts here")
        self.toplabel.pack()

        self.labelframe2 = LabelFrame(self.f, text = "Changes You want!!")
        self.labelframe2.pack(fill="both", expand = "yes")

        self.bottomlabel = Label(self.labelframe2, text = "You can put here the changes you want,If any!")
        self.bottomlabel.pack()
```

```
  root= Tk()
lf=MyLabelFrame(root)
root.mainloop()
```

- **Menu Widgets-**
  - Menu represent a group of items or options for user to select.
  - Ex. File menu, edit menu etc.
  - Generally menu is displayed in bar called as menu bar.
  - To create menu we should use following steps
  - First we should create menu bar as a child to root window by creating object of **Menu** class as
  - ***menubar=Menu(root)***
  - This menu should be attach to root window using config() method as
  - ***root.config(menu=menubar)***
  - Then create submenus as
  - ***filemenu=Menu(root, tearoff=0)***
  - Here traroff can either 0 or 1. when value is 1, the menu can be torn off i.e. the menu item is occupied by the tearoff element which is dashed line. And value 0, then this dashed line will not apper and menu item will displayed starting from 0th position.

- **Menu Widgets-**
  - The next step is ass menu items to sub menu (ex. Filemenu) using add_command() method as
  - ***filemenu.add_command(label="New", command=donothing)***
  - Here we add sub menu New and when user click on this menu donothing() method will be called.
  - When we want to add horizontal line that seprates group of menu items, we can use add_separator() as
  - ***filemenu.add_separator()***
  - After adding all submenus to menu(filemenu) we should give its name and add it to menu bar using add_cascade() method as
  - ***Menubar.add_cascade(label="File", menu=filemenu)***
  - Here menu option tells that the "File" menu is composed of all submenus that are already added to filemenu.

- **Menu Widgets-**

Name of the Option	Description
activebackground	This option is used to indicate the background color of the widget when the widget is under the focus.
activeforeground	This option indicates the font color of text of the widget when the widget has the focus.
activeborderwidth	This option is used to indicate the width of the border of the widget when it is under the mouse(when it is active). The default value of this option is 1 pixel.
bd	This option is used to indicate the border width of the widget
bg,fg	This option indicates the background and foreground color of the widget.
cursor	This option indicates the cursor when the mouse hovers the menu.
disabledforeground	This option indicates the text color of the widget when the widget is disabled
font	This option is used to indicate the font type of the text of widget
relief	This option is used to specify the border type. Its default value is RAISED.
image	This option is used to display an image on the menu
postcommand	This option can be set to any of the function which is called when the mouse hovers the menu.
tearoff	The choices in the menu start from position 1 by default. But If we set the tearoff=1, then choices will start taking place from 0th position.
selectcolor	This option indicates the color used to display the checkbutton or radiobutton when they are selected.
title	This option is set to the title of the window if you want to change the title of the window.

- **Menu Widgets-Methods**

Name of method	Description
add_command()	This method is used to add menu items to the menu.
add_radiobutton()	This method is used to add the radiobutton to the menu.
add_checkbutton()	This method is mainly used to add checkbuttons to the menu.
add_cascade()	This method is used to create a hierarchical menu to the parent menu by associating the given menu to the parent menu.
add_seperator()	This method is used to add the separator line to the menu items.
add(type, options)	This method is used to add the specific menu item to the menu.
delete(startindex, endindex)	This method is used to delete the menu items that exist in the specified range.
entryconfig(index, options)	This method is used to configure a menu item that is identified by the given index.
index(item)	This method is used to get the index of the specified menu item.
insert_seperator(index)	This method is used to insert a separator at the specified index.
invoke(index)	This method is used to invoke the associated operation with the choice given at the specified index.
type(index)	This method is used to get the type of choice specified by the index.

- **Menu Widgets-**

```
from tkinter import *
class MyMenu:
    def __init__(self,root):
        self.menubar=Menu(root)
        root.config(menu=self.menubar)

        self.filemenu=Menu(root,tearoff=0)
        self.filemenu.add_command(label="New",
command=self.donothing)
        self.filemenu.add_command(label="Open",
command=self.donothing)
        self.filemenu.add_command(label="Save",
command=self.donothing)
        self.filemenu.add_separator()
        self.filemenu.add_command(label="Exit",
command=self.donothing)

self.menubar.add_cascade(label="File",menu=self.filemen
u)
```

```
 self.editmenu=Menu(root,tearoff=0)
        self.editmenu.add_command(label="Cut",command=self.donothing)
        self.editmenu.add_command(label="Copy",command=self.donothing)
        self.editmenu.add_command(label="Paste",command=self.donothing)
        self.editmenu.add_separator()
        self.editmenu.add_command(label="Find",command=self.donothing)

self.editmenu.add_command(label="Replace",command=self.donothing)
        self.menubar.add_cascade(label="Edit",menu=self.editmenu)


    def donothing(self):
        pass

root=Tk()
mm=MyMenu(root)
root.geometry("600x500")
root.title("My Menu Application")
root["bg"]="light gray"
root.mainloop()
```

- **Menubutton Widgets-**
  - Menubutton widget is used to create a dropdown menu which is attached to button
  - This widget is used to provide various types of menus in the Python Application.
  - It is important to note that every Menubutton in an application is associated with a Menu widget and that in return can display the choices for that menubutton whenever the user clicks on it.
  - The Tkinter Menubutton widget provides the user with an option to select the appropriate choice that exists within the application.
  - It is created by creating obect of MenuBitton class
  - ***W = Menubutton(master, options)***
  - In the above syntax, the master parameter denotes the parent window.
  - You can use many options to change the look of the menubuttons and these options are written as comma-separated key-value pairs.

- **Menubutton Widgets-**

Option name	Description
activebackground	This option indicates the background color of the menubutton at the time when the mouse hovers the menubutton.
bd	This option is used to represent the width of the border in pixels. The default value is 2 pixels.
bitmap	This option will be set to the graphical content which is to be displayed to the widget.
bg	This option is used to represent the background color of the widget.
cursor	This option indicates the cursor when the mouse hovers the menubutton.
activeforeground	This option mainly represents the font color of the widget at the time when the widget is under the focus
fg	This option represents the foreground color of the widget.
direction	With the help of this option, you can specify the direction so that menu can be displayed to the specified direction of the button. You can Use LEFT, RIGHT, or ABOVE to place the widget accordingly.
disabledforeground	This option indicates the text color of the widget when the widget is disabled
height	This option indicates the height of the menubutton. This height indicates the number of text lines in the case of text lines and it indicates the number of pixels in the case of images.
image	This option indicates the image displayed on the menubutton.
higlightcolor	This option indicates the highlight color when there is a focus on the button
justify	This option is used to indicate the way by which the multiple text lines are represented. For left justification, it is set to LEFT and it is set to RIGHT for the right justification, and CENTER for the center justification.

- **Menubutton Widgets-**

Option name	Description
padx	This option indicates the additional padding of the widget in the horizontal direction.
pady	This option indicates the additional padding of the widget in the vertical direction.
menu	This option is used to indicate the menu associated with the menubutton
width	This option specifies the width of the widget. For textual buttons, It exists as a number of letters or for image buttons it indicates the pixels
Wraplength	In this case, if this option's value is set to a positive number, the text lines will be wrapped in order to fit within this length.
state	As the normal state of menubutton is enabled.It can be set to disable to make the menubutton unresponsive.
text	This option is used to indicate the text on the widget.
textvariable	A control variable of class StringVar can be associated with this menubutton. If you will set that control variable then it will change the displayed text.
underline	This option is mainly used to represent the index of the character in the text of the widget which is to be underlined. The indexing generally starts with zero in the text.
relief	This option is used to specify the border type. Its default value is RAISED

- **Menubutton Widgets-**

```
import tkinter

win = Tk()

mbtn = Menubutton(win, text="Courses", relief=RAISED)
mbtn.grid()
mbtn.menu = Menu(mbtn, tearoff = 0)
mbtn["menu"] = mbtn.menu

pythonVar = IntVar()
javaVar = IntVar()
phpVar = IntVar()

mbtn.menu.add_checkbutton(label="Python",
variable=pythonVar)
mbtn.menu.add_checkbutton(label="Java",
variable=javaVar)
mbtn.menu.add_checkbutton(label="PHP",
variable=phpVar)

mbtn.pack()
win.mainloop()
```

- **Filedialog-**
  - When developing a Tkinter application that deals with the file system, you need to provide a dialog that allows file selections.
  - To do that, you can use the ***tkinter.filedialog*** module.
  - The following steps show how to display an open file dialog:
  - First, import the tkinter.filedialog module:
  - ***from tkinter import filedialog as fd***
  - Second, call the fd.askopenfilename() function to show a dialog that allows a single file selection:
  - ***filename = fd.askopenfilename()***
  - The ***askopenfilename()*** function returns the file name that you selected.
  - The askopenfilename() also supports other useful options including the initial directory displayed by the dialog or filtering files by their extensions.
  - The ***askopenfilenames()*** function displays a file dialog for multiple file selections. It returns the selected file names as a tuple.

- **Filedialog-**
  - After getting the selected file names, you can open them using the open() method.
  - To make it more convenient, the tkinter.filedialog module also provides some functions that allow you to select one or more files and return the file objects directly.
  - The ***askopenfile()*** function displays a file dialog and returns a file object of the selected file
  - ***f = fd.askopenfile()***
  - The ***askopenfiles()*** function displays a file dialog and returns a file object of the selected files
  - ***f = fd.askopenfiles()***

- **Filedialog**

```
import tkinter as tk
from tkinter import ttk
from tkinter import filedialog as fd

# Root window
root = tk.Tk()
root.title('Display a Text File')
root.resizable(False, False)
root.geometry('550x250')

# Text editor
text = tk.Text(root, height=12)
text.grid(column=0, row=0, sticky='nsew')


def open_text_file():
    # file type
    filetypes = (
        ('text files', '*.txt'),
        ('All files', '*.*')
    )
```

```
# show the open file dialog
    f = fd.askopenfile(filetypes=filetypes)
    # read the text file and show its content on the Text
    text.insert('1.0', f.readlines())


# open file button
open_button = ttk.Button(
    root,
    text='Open a File',
    command=open_text_file
)

open_button.grid(column=0, row=1, sticky='w', padx=10,
pady=10)


root.mainloop()
```

- **Color Chooser Module–**
  - To display a native color chooser dialog, you use the tkinter.colorchooser module.
  - First, import the askcolor() function from the tkinter.colorchooser module:
  - ***from tkinter.colorchooser import askcolor***
  - Second, call the askcolor() function to display the color chooser dialog:
  - ***askcolor(color=None, **options)***
  - If you select a color, the askcolor() function returns a tuple that contains two values that represent the selected color:
    - The first value is the RGB representation.
    - The second value is a hexadecimal representation.
    - Ex. ((255.99609375, 0.0, 255.99609375), '#ff00ff')

- **Color Chooser Module-**

```
import tkinter as tk
from tkinter.colorchooser import askcolor


root = tk.Tk()
root.title('Tkinter Color Chooser')
root.geometry('300x150')


def change_color():
    colors = askcolor(title="Tkinter Color Chooser")
    root.configure(bg=colors[1])


ttk.Button(
    root,
    text='Select a Color',
    command=change_color).pack(expand=True)


root.mainloop()
```

- **Messagebox Module–**
  - When developing a Tkinter application, you often want to notify users about the events that occurred.
  - For example, when users click the save button, you want to notify them that the record has been saved successfully.
  - If an error occurred, for example, the database server is not reachable, you can notify users of the error.
  - When the update has been completed but the record already exists, you may want to show a warning.
  - To cover all of these scenarios, you can use various functions from the tkinter.messagebox module:
    - showinfo() – notify that an operation completed successfully.
    - showerror() – notify that an operation hasn't completed due to an error.
    - showwarrning() – notify that an operation completed but something didn't behave as expected.

- **Messagebox Module–**
  - All of these functions accept two arguments:
  - ***showinfo(title, message)***
  - ***showerror(title, message)***
  - ***showwarrning(title, message)***
  - The title is displayed on the title bar of the dialog.
  - The message is shown on the dialog.

- **Messagebox Widgets-**

```
import tkinter as tk
from tkinter import ttk
from tkinter.messagebox import showerror, showwarning,
showinfo

# create the root window
root = tk.Tk()
root.title('Tkinter MessageBox')
root.resizable(False, False)
root.geometry('300x150')

#
options = {'fill': 'both', 'padx': 10, 'pady': 10, 'ipadx': 5}

ttk.Button(
    root,
    text='Show an error message',
    command=lambda: showerror(
        title='Error',
        message='This is an error message.')
).pack(**options)
```

```
ttk.Button(
    root,
    text='Show an information message',
    command=lambda: showinfo(
        title='Information',
        message='This is an information message.')
).pack(**options)


ttk.Button(
    root,
    text='Show an warning message',
    command=lambda: showwarning(
        title='Warning',
        message='This is a warning message.')
).pack(**options)


# run the app
root.mainloop()
```

- **askyesno Dialog–**
  - Sometimes, you need to ask for user confirmation.
  - For example, if users click the quit button, you want to ask whether they really want to close the application. Or they just accidentally do so:
  - To show a dialog that asks for user confirmation, you use the askyesno() function.
  - The dialog will have a title, a message, and two buttons (yes and no).
  - When you click the yes button, the function returns True. However, if you click the no button, it returns False.
  - The following shows the syntax of the askyesno() function:
  - ***answer = askyesno(title, message, **options)***
  - answer is a Boolean value, either True or False.
  - Tkinter also has another function called askquestion(), which is similar to the askyesno() function except that it returns a string with a value of 'yes' or 'no‘
  - ***answer = askquestion(title, message, **options)***

- **askyesno Dialog–**
  - The askokcancel() function shows a confirmation dialog that has two buttons: OK and Cancel.
  - ***answer = askokcancel(title, message, **options)***
  - Sometimes, the application performs a task but fails to do so because of an error.
  - For example, you may want to connect to a database server. However, the database server is currently not reachable. It may be offline for a short period of time.
  - In this case, you can display a confirmation dialog that allow users to reconnect to the database or just keep the application as is.
  - To display the Retry/Cancel dialog, you can use the askretrycancel() function:
  - ***answer = askretrycancel(title, message, **options)***
  - The askretrycancel() function returns True if the Retry button is clicked. If the Cancel button is clicked, it returns False.

- **askyesno type Dialog-**

```
import tkinter as tk
from tkinter import ttk
from tkinter.messagebox import askretrycancel, showinfo

# create the root window
root = tk.Tk()
root.title('Tkinter OK/Retry Dialog')
root.geometry('300x150')

# click event handler
def confirm():
    answer = askretrycancel(
        title='Connection Issue',
        message='The database server is unreachable. Do
you want to retry?'
    )
    if answer:
        showinfo(
            title='Information',
            message='Attempt to connect to the database
again.')
```

```
ttk.Button(
    root,
    text='Connect to the Database Server',
    command=confirm).pack(expand=True)


# start the app
root.mainloop()
```

- **Creating Table-**
  - A table is useful to display data in the form of rows and columns
  - But python doesn't provide table widget but we can create table using alternate method such as using Entry widget or Label widget.
  - For creating table we need to use tow loops i.e. one for row and one for column
  - Then inside the loop we have to create Entry/Label widget to display data
  - Now we need to place these widgets at proper row and column position so for this purpose we used grid() geometry layout method.

- **Creating Table-**

```
from tkinter import *
class MyTable:
    def __init__(self,root):


        lst=[(1001,"Sunil Patil","BCS-I",67.56),(1002,"Arati
Mane","BCS-II",89.6),(1003,"Vijay
Sathe","BCS-II",86.76),(1004,"Satish
Shinde","BCA-I",78.9)]
        rows=len(lst)
        cols=len(lst[0])

        for i in range(rows):
            for j in range(cols):
                self.txt=Entry(root,width=20, fg="blue",
font=("Georgia",15,"bold"))
                self.txt.grid(row=i,column=j)
                self.txt.insert(END, lst[i][j])
```

```
root=Tk()
mt=MyTable(root)
root.mainloop()
```

- **Evet Binding**
  - Tkinter provides an alternative form of an event binding mechanism called bind() to let you deal with different events.
  - The standard syntax used to bind an event is as follows:
  - ***widget.bind(event, handler, add=None)***
  - When an event corresponding to the event description occurs in the widget, it calls not only the associated handler that passes an instance of the event object as the argument, but also the details of the event.
  - If there already exists a binding for that event for this widget, the old callback is usually replaced with the new handler, but you can trigger both the callbacks by passing add='+' as the last argument.

- **Evet Binding**

```
from tkinter import *
root = Tk()
Label(root, text='Click at different\n locations in the frame below').pack()
def callback(event):
    print (dir(event))
    print ("you clicked at", event.x, event.y)
frame = Frame(root, bg='khaki', width=130, height=80)
frame.bind("<Button-1>", callback)
frame.pack()
root.mainloop()
```

- We bind the Frame widget to the <Button-1> event, which corresponds to the left-click.
- When this event occurs, it calls the callback function, passing an object instance as its argument.
- We define the callback(event) function.
- Note that it takes the event object generated by the event as an argument.
- We inspect the event object by using dir(event), which returns a sorted list of attribute names for the event object passed to it.
- From the attributes list generated by the object, we use two attributes, event.x and event.y, to print the coordinates of the point of click.

- **Event patterns**
  - In the previous example, you learned how to use the <Button-1> event to denote a left-click.
  - This is a built-in pattern in Tkinter that maps it to a left-click event.
  - Tkinter has an exhaustive mapping scheme that perfectly identifies events such as this one.
  - Here are some examples to give you an idea of event patterns:

The event pattern	The associated event
<Button-1>	Left-click of the mouse
<KeyPress-B>	A keyboard press of the B key
<Alt-Control-KeyPress- KP_Delete>	A keyboard press of Alt + Ctrl + Delete

- **Event patterns**
  - In general, the mapping pattern takes the following form:
  - ***<[event modifier-]...event type [-event detail]>***
  - Typically, an event pattern will comprise the following:
  - An event type:
    - Some common event types include Button, ButtonRelease, KeyRelease, Keypress, FocusIn, FocusOut, Leave (when the mouse leaves the widget), and MouseWheel.
    - For a complete list of event types, refer to the The event types section at http://www.tcl.tk/man/tcl8.6/TkCmd/bind.htm#M7.
  - An event modifier (optional):
    - Some common event modifiers include Alt, Any (used like <Any-KeyPress>), Control, Double (used like <Double-Button-1> to denote a double-click of the left mouse button), Lock, and Shift.
    - For a complete list of event modifiers, refer to the The event modifiers section at http://www.tcl.tk/man/tcl8.6/TkCmd/bind.htm#M6.

- **Event patterns**
  - The event detail (optional):
    - The mouse event detail is captured by the number 1 for a left-click and the number 2 for a right-click.
    - Similarly, each key press on the keyboard is either represented by the key letter itself (say, B in <KeyPress-B>) or by using a key symbol abbreviated as keysym.
    - For example, the up arrow key on the keyboard is represented by the keysym value of KP_Up.
    - For a complete keysym mapping, refer to https://www.tcl.tk/man/tcl8.6/TkCmd/bind.htm.

GUI in Python-Evet Binding

- **Event patterns**
  - The following is a modified snippet of code; it will give you an idea of the commonly used event bindings:
  - widget.bind("<Button-1>", callback) #bind widget to left mouse click
  - widget.bind("<Button-2>", callback) # bind to right mouse click
  - widget.bind("<Return>", callback) # bind to Return(Enter) Key
  - widget.bind("<FocusIn>", callback) #bind to Focus in Event
  - widget.bind("<KeyPress-A>", callback) # bind to keypress A
  - widget.bind("<KeyPress-Caps_Lock>", callback) # bind to CapsLock keysym
  - widget.bind("<KeyPress-F1>", callback) # bind widget to F1 keysym
  - widget.bind("<KeyPress-KP_5>", callback) # bind to keypad number 5
  - widget.bind("<Motion>", callback) # bind to motion over widget
  - widget.bind("<Any-KeyPress>", callback) # bind to any keypress

- **Event unbinding and virtual events**
  - In addition to the bind method that you previously saw, you might find the following two event-related options useful in certain cases:
  - unbind:
    - Tkinter provides the unbind option to undo the effect of an earlier binding. The syntax is as follows:
      - ***widget.unbind(event)***
    - The following are some examples of its usage:
      - ***entry.unbind('<Alt-Shift-5>')***
      - ***root.unbind_all('<F1>')***
      - ***root.unbind_class('Entry', '<KeyPress-Del>')***

- **Event unbinding and virtual events**
  - Virtual events:
    - Tkinter also lets you create your own events. You can give these virtual events any name that you want.
    - For example, let's suppose that you want to create a new event called
      - ***<<commit>>,*** which is triggered by the F9 key.
    - To create this virtual event on a given widget, use the following syntax:
      - ***widget.event_add('<<commit>>', '<KeyRelease-F9>')***
  - You can then bind <<commit>> to a callback by using a normal bind() method, as follows:
      - ***widget.bind('<<commit>>', callback)***
  - Other event-related methods can be accessed by typing the following line in the Python terminal:
  - ***import tkinter***
  - ***help(tkinter.Event)***

python

Database Connection

Introduction

- When data is stored in a database like MySQL or Oracle, we can retrieve the data using the Python program i.e our python program is able to connect various databases.
- To connect to the database we need a driver or connector program that connect the database to python program
- Here we can focus on MySQL database.
- Before going to connectivity program we must check whether MySQL is installed in the system
- You may download MySQL form mysql site and install it.
- After successful installation of MySQL we need to download and install driver or connector which is useful to connect database to python program.
- ***"MySQLdb"*** is a interface/driver/connector between MySQL and Python program.

Working MySQL with Python

- To install these driver you need to download using following path
- ***https://pypi.python.org/pypi/mysqlclient***
- Downloaded file name is ***mysqlcient-1.3.12-cp36-cp36m-win_amd64.whl***
- After downloading this file use following command to install
- ***pip install mysqlcient-1.3.12-cp36-cp36m-win_amd64.whl***
- Or you can directly use following command to install ***MySQLdb***
- ***pip install mysqlclient***
- To work with MySQL in python program, we have to use MySQLdb module and this will be achieving b importing the module as
- ***import MySQLdb or***
- ***import MySQLdb as mysql***

Working MySQL with Python

- You should aware that MySQLdb module depends upon a module called _mysql.
- _mysql module is largely a python adaptation of MySQL C API and it is this API which will access through MySQL for python.
- _mysql module is powerful and proven way of accessing MySQL database.
- You can access documentation for _mysql by using MySQLdb module as
- ***help(MySQLdb._mysql)***

Working MySQL with Python

- After importing module we verify or access documentation of module as
- ***help(MySQLdb)***
- This help shows some of the important packages as
  - ***connections***-initiating, maintaining and closing of MySQL connection
  - ***cursors***-managing the executions of queries
  - ***converters***- converting MySQL data types to python
  - ***times***-converting date and time values between MySQL and python
- The most important part of module is connections.py without which we could not interface MySQL
- ***help(MySQLdb)***
- When a connection or operational request is received by connections.Connection(), it is processed and passed to _mysql and subsequently to the MySQL C API to perform it.

Working MySQL with Python

- To handle this interface, _mysql uses two classes
  - Connection- it is used to establish communication with MySQL and return connection object.
  - Result- returns a set containing the result from MySQL command. The result ma be quer result or and error.
- Making a database connection through MySQL for python consist of 4 stages
  - Creating a connection object
  - Creating a cursor object
  - Interacting with database
  - Closing the connection

Creating connection Object

- To establish connection with MySQL database in python program, we use connect() method of MySQLdb module as
- ***Conn=MySQLdb.connect(host="localhost", user="root", password="root", database="pythondb")***
- Here host represent server name or IP address. If we use ***standalone MySQL*** then it will be ***"localhost"*** and if we use ***Server based MySQL*** then it will be ***IP address or server name***
- User and password represent username and password of MySQL
- Database represent name of database to connect
- Conn represent object of MySQL connection class.

Creating Cursor Object

- After connection object is created, you cannot interact with the database until you create cursor object.
- To create cursor we use cursor() method of MySQLdb.connection object we created for the connection as
- ***[cursor_name]=[connection_object_name].cursor() #Syntax***
- ***cursor=conn.cursor() #Example***
- Now we are ready to issue the command

Interact with Database

- Cursor object are useful to execute SQL commands on the database.
- For this purpose we use execute() method of cursor object as
- ***cursor.execute("select * from dept")***
- The resultant rows are retrieved from the table dept and stored in cursor object.
- We can get records from the cursor object using fetchone() and fetchall() methods
  - ***fetchone()***- get single row
  - ***fetchall()***- get all rows

Close the connection

- Finally we can close the connection with MySQL by closing the cursor and connection object.
- To do this weuse close() method.
- ***cursor.close()***
- ***conn.close()***
- Before going to close the connection you need to check whether database auto-commit is on or not.
- If auto-commit is not on then you will need to commit and for this purpose you may use commit() of connection object as
- ***conn.commit()***
- After committing all the changes we can close the connection.